Penser avant de générer.

Apprendre le Web quand on part de zéro : HTML, CSS, JavaScript, Git et GitHub. Un guide pratique pour comprendre ce que l'on construit, à l'ère où l'IA peut écrire du code à votre place.

01

DAVID MBULA WAWINA

DÉVELOPPEUR FULL-STACK · CONSULTANT DIGITAL · WEBMASTER

L'AVANTAGE DU DEVELOPPEUR · TOME 01

Penser avant de générer

Apprendre le Web quand on part de zéro : HTML, CSS, JavaScript, Git et GitHub. Un guide pratique pour comprendre ce que l'on construit, à l'ère où l'IA peut écrire du code à votre place.

DAVID MBULA WAWINA

DÉVELOPPEUR FULL-STACK · CONSULTANT DIGITAL · WEBMASTER

Penser avant de générer

L'avantage du développeur, Tome 01 sur 3

Première édition numérique · octobre 2026 · version 1.0

© 2026 David Mbula Wawina. Tous droits réservés.

Ce livre numérique est destiné à l'usage personnel de son acheteur. Toute reproduction, diffusion, revente ou mise à disposition, totale ou partielle, sans l'autorisation écrite de l'auteur est interdite.

Les exemples de code sont fournis à des fins pédagogiques. Vous pouvez les réutiliser librement dans vos propres projets.

Les noms de logiciels, services et marques cités appartiennent à leurs propriétaires respectifs. Les liens ont été vérifiés en octobre 2026 ; un service en ligne peut changer d'adresse ou de fonctionnement après cette date.

Questions et retours : WhatsApp · davidwawina.site

TOME 01

Sommaire

Introduction 6

Votre environnement de travail 10

HTML 18

Comprendre HTML 19

Organiser une vraie page HTML 25

Mini-projet HTML : votre page de présentation 30

CSS 34

Comprendre CSS 35

Le box model et la mise en page 40

Mini-projet CSS : un portfolio professionnel 45

JavaScript 50

Comprendre JavaScript 51

Le DOM et les événements 56

Mini-projet JavaScript : un portfolio interactif 61

Git 66

Qu'est-ce que Git ? 67

Les commandes Git essentielles 71

GitHub 75

Comprendre GitHub 76

Branches et pull requests, simplement 81

Apprendre avec l'IA 84

Apprendre avec l'IA sans perdre votre avantage 85

Ressources pour continuer 89

Votre parcours de 30 jours 91

Projet final — Votre premier portfolio 92

Conclusion — Commencez aujourd'hui 94

Annexe A — Glossaire 95

Annexe B — Quand ça ne marche pas 96

Introduction

Ce livre est pour vous, même si vous n'avez jamais écrit une seule ligne de code.

Vous avez peut-être déjà entendu cette phrase : « Aujourd'hui, l'intelligence artificielle écrit le code toute seule, alors à quoi bon apprendre ? » C'est une question légitime. Des outils comme ChatGPT, Claude, Gemini ou GitHub Copilot sont capables de produire une page Web en quelques secondes. Il suffit de leur demander.

Mais regardez ce qui se passe ensuite. La page s'affiche. Puis vous voulez changer une couleur, déplacer un bouton, corriger un formulaire qui ne s'envoie pas. Vous redemandez. L'outil propose une nouvelle version, qui casse autre chose. Vous redemandez encore. Au bout d'une heure, vous avez un fichier de trois cents lignes que vous ne comprenez pas, qui fonctionne à moitié, et que vous n'osez plus toucher.

Ce scénario a un nom : **la boîte noire**. Vous obtenez un résultat sans comprendre comment il a été produit. Tant que tout fonctionne, la boîte noire paraît magique. Dès que quelque chose casse, elle devient une prison.

Ce livre vous propose l'inverse. Il vous apprend à **penser avant de générer** : comprendre ce qu'est une page Web, comment elle est construite, comment elle réagit, comment on la sauvegarde et comment on la publie. Une fois ces bases acquises, l'IA ne sera plus une boîte noire. Elle deviendra un assistant que vous pouvez diriger, corriger et contredire.

> **LA REGLE DU LIVRE**
>
> Ne copiez pas seulement le code. **Écrivez-le**, modifiez-le, cassez-le, observez le résultat et essayez de l'expliquer avec vos propres mots. Un code que vous avez tapé vous-même, même lentement, vous apprend dix fois plus qu'un code collé.

Un livre volontairement recentré

Ce livre ne cherche pas à vous faire découvrir dix langages, des Frameworks à la mode ou des architectures de serveur complexes. Son objectif est plus simple et plus

durable : vous apprendre à comprendre et à utiliser les **cinq briques** qui constituent une excellente porte d'entrée vers le développement Web.

BRIQUE	SON ROLE	IMAGE POUR S'EN SOUVENIR
HTML	Structurer le contenu de la page	Le squelette et les pièces d'une maison
CSS	Définir l'apparence	La peinture, les meubles, la décoration
JavaScript	Donner un comportement	L'électricité : les interrupteurs, les lumières
Git	Garder l'historique de votre travail	Un album photo de chaque étape du chantier
GitHub	Sauvegarder, partager, publier	La vitrine et le coffre-fort en ligne

Ces cinq outils sont utilisés chaque jour par des millions de développeurs professionnels. Ils ne passeront pas de mode demain : ce que vous apprendrez ici restera utile pendant des années, y compris lorsque vous travaillerez avec l'IA.

Ce dont vous n'avez pas besoin

Vous n'avez pas besoin d'être fort en mathématiques. Le développement Web demande surtout de la logique, de la patience et de l'organisation, pas des équations.

Vous n'avez pas besoin de parler parfaitement anglais. Les mots-clés du code sont en anglais, mais ils sont peu nombreux et vous les apprendrez naturellement : `title` signifie titre, `button` signifie bouton, `color` signifie couleur. Ce livre vous explique chacun d'eux.

Vous n'avez pas besoin d'un ordinateur puissant, ni même d'un ordinateur tout court pour commencer. Un smartphone suffit pour les premières semaines. Le chapitre 1 vous explique comment faire.

Ce dont vous avez vraiment besoin, c'est de **pratiquer régulièrement**, de **comprendre ce que vous écrivez** et d'**accepter de faire des erreurs**. Chaque développeur, même le plus expérimenté, passe une partie de sa journée à corriger des erreurs. Ce n'est pas un signe d'échec : c'est le métier.

Ce que vous serez capable de faire

À la fin de ce livre, vous saurez :

- créer une page Web structurée avec **HTML** ;
- la rendre agréable à regarder et adaptée aux téléphones avec **CSS** ;

- lui ajouter des interactions avec **JavaScript** : boutons, formulaires, thème clair et sombre ;
- organiser vos projets et suivre leurs modifications avec **Git** ;
- sauvegarder, partager et **publier en ligne** vos projets avec **GitHub** ;
- commencer depuis un smartphone, puis travailler confortablement sur Windows ou Mac ;
- utiliser l'IA comme un professeur patient, sans la laisser penser à votre place.

Et surtout, vous aurez construit et publié **votre premier portfolio** : une vraie page en ligne, à votre nom, que vous pourrez montrer et que vous saurez expliquer ligne par ligne.

Comment utiliser ce livre

Chaque chapitre suit la même logique :

définition simple → explication → exemple → pratique → exercices

Faites les exercices avant de passer au chapitre suivant. Ils sont courts et progressifs, et ils construisent tous le même projet : votre portfolio. Si vous sautez les exercices, vous aurez lu un livre. Si vous les faites, vous aurez appris un métier.

Si une notion vous semble difficile, revenez à l'exemple et **changez une seule chose à la fois**. Modifiez une couleur, rechargez la page, observez. Modifiez un mot, rechargez, observez. Cette méthode, simple et un peu lente, est la plus efficace qui existe pour comprendre du code.

Vous rencontrerez plusieurs types d'encadrés :

> **À RETENIR**
> Les idées essentielles d'une section, à relire avant de passer à la suite.

> **ATTENTION**
> Une erreur fréquente ou un piège dans lequel presque tous les débutants tombent.

> **AVEC L'IA**
> Comment utiliser un assistant d'intelligence artificielle **intelligemment** à cette étape, sans perdre votre avantage.

> **EXERCICES**
> La pratique. C'est la partie la plus importante de chaque chapitre.

Combien de temps faut-il ?

Le livre se termine par un **parcours de 30 jours**. Avec une heure de pratique par jour, vous pouvez le suivre en un mois. Si vous avez moins de temps, prenez-en deux ou trois : l'important n'est pas la vitesse, c'est la régularité. Une demi-heure chaque jour vaut mieux que cinq heures le dimanche.

> **? VOUS ETES BLOQUE ? POSEZ VOTRE QUESTION**
>
> Vous ne serez pas seul. Si un exercice vous bloque, si une installation échoue ou si une notion reste floue, vous pouvez m'écrire directement sur WhatsApp : poser une question à David.
>
> Pour recevoir une réponse rapide et utile, précisez toujours **le chapitre**, **ce que vous essayez de faire**, **ce qui se passe à la place** et, si possible, une capture d'écran de votre code et du résultat. Apprendre à bien poser une question est déjà une compétence de développeur.

Prêt ? Commençons par installer votre poste de travail.

CHAPITRE 01

Votre environnement de travail

Avant d'écrire du code, il faut un endroit pour l'écrire et un endroit pour voir le résultat. Rien de plus.

1.1 De quoi avez-vous besoin ?

Pour commencer, il vous faut trois choses :

1. **un appareil pour écrire du code** : un ordinateur ou un smartphone ;
2. **un navigateur pour voir le résultat** : Chrome, Firefox, Edge ou Safari ;
3. **un peu de temps chaque jour** pour pratiquer.

Une page Web, à la base, n'est qu'un **fichier texte** avec une extension particulière : `.html`. Le navigateur lit ce fichier et le transforme en page visuelle. Vous n'avez besoin d'aucun logiciel payant, d'aucun serveur et d'aucune connexion Internet permanente pour faire vos premiers pas.

Un ordinateur est idéal, mais un smartphone peut parfaitement servir pour les premières étapes. Voyons les deux.

1.2 Commencer avec un smartphone

Si vous n'avez pas encore d'ordinateur, ne vous arrêtez pas là. Plusieurs applications permettent d'écrire du HTML, du CSS et du JavaScript directement sur un téléphone, et d'afficher le résultat.

Sur Android

APPLICATION	CE QU'ELLE OFFRE	LIEN
Acode	Éditeur de code complet : explorateur de fichiers, coloration du code, prévisualisation, terminal	Google Play · guide officiel
TrebEdit	Pensé pour le Web : éditeur HTML, visionneuse intégrée, console JavaScript	Google Play
Replit	Créer, exécuter et publier des projets Web, avec un compte en ligne	replit.com/mobile

Pour ce livre, nous recommandons **Acode** : il est gratuit, fonctionne hors ligne et ressemble aux éditeurs utilisés sur ordinateur, ce qui facilitera votre passage au PC plus tard.

Sur iPhone et iPad

APPLICATION	CE QU'ELLE OFFRE	LIEN
Textastic	Éditeur de code avec prévisualisation HTML. Téléchargement gratuit, fonctions avancées payantes	App Store
Replit	Même service que sur Android : projets Web en ligne avec un compte	replit.com/mobile
Code App	Éditeur très complet inspiré de VS Code, avec terminal et Git. Application payante, surtout confortable sur iPad	App Store

> **LES PRIX CHANGENT**
>
> Les conditions des applications (gratuites, payantes, abonnements) évoluent avec le temps. Vérifiez toujours le prix affiché sur la fiche de l'App Store ou de Google Play avant d'installer. Pour débuter, une solution gratuite suffit largement.

Directement dans le navigateur, sans rien installer

Quel que soit votre téléphone, vous pouvez aussi coder dans le navigateur :

- **CodePen**affiche trois zones (HTML, CSS, JavaScript) et le résultat en direct. Idéal pour tester une idée en deux minutes.
- **github.dev**est l'éditeur en ligne de GitHub. Il servira plus tard, quand vous aurez un compte GitHub (chapitre 13).

> **TELEPHONE OU ORDINATEUR ?**
>
> Oui, vous pouvez commencer sur téléphone. Mais dès que possible, passez sur un ordinateur, même ancien ou emprunté. Un clavier physique, un grand écran, plusieurs fenêtres côte à côte, une vraie gestion des fichiers, Git et les outils de développement du navigateur rendent l'apprentissage beaucoup plus confortable et rapide. Le téléphone est une rampe de lancement, pas un poste de travail définitif.

1.3 Coder sur téléphone : la méthode pas à pas

Voici la démarche avec Acode. Elle est très proche dans les autres applications.

1. **Installez Acode** depuis Google Play et ouvrez-le.
2. **Créez un dossier de projet.** Dans le menu, ouvrez l'explorateur de fichiers, choisissez un emplacement (par exemple le stockage interne) et créez un dossier nommé `premier-site`.
3. **Créez un fichier** dans ce dossier et nommez-le exactement `index.html`. Le nom `index.html` est une convention : c'est la page d'accueil d'un site.
4. **Écrivez quelques lignes**, par exemple :

```
<h1>Bonjour, je commence le développement Web !</h1>
<p>Ceci est ma toute première page.</p>
```

1. **Enregistrez** le fichier (icône de disquette ou menu *Enregistrer*).
2. **Affichez le résultat** avec le bouton de prévisualisation (souvent une icône ▶ ou un œil). Vous devez voir un grand titre et une phrase.
3. **Modifiez une chose**, enregistrez, rechargez et observez. Remplacez *Bonjour* par votre prénom, par exemple.

Plus tard, vous ajouterez dans le même dossier un fichier `style.css` pour l'apparence et un fichier `script.js` pour les interactions. Les trois fichiers vivront ensemble, comme sur un ordinateur.

> **À RETENIR**
>
> Un projet Web est un **dossier** qui contient des **fichiers texte**. Le fichier `index.html` est la porte d'entrée. Le navigateur lit ce fichier et affiche la page.

1.4 Sur Windows ou Mac

Sur ordinateur, une configuration simple suffit :

- Windows 10 ou 11, ou macOS ;
- Un navigateur moderne : **Google Chrome**, **Firefox** ou **Microsoft Edge** ;
- **Visual Studio Code**, l'éditeur de code que nous utiliserons comme référence dans ce livre.

Visual Studio Code, souvent appelé **VS Code**, est gratuit, disponible sur Windows, macOS et Linux, et utilisé par une très grande partie des développeurs Web dans le monde. Ne le confondez pas avec *Visual Studio* tout court, qui est un autre logiciel, plus lourd, destiné à d'autres usages.

Git, l'outil qui garde l'historique de votre code, sera installé plus tard, au chapitre 11, au moment où vous comprendrez à quoi il sert. Pour l'instant, VS Code et un navigateur suffisent.

Installer VS Code sur Windows

1. Rendez-vous sur code.visualstudio.com et cliquez sur le bouton de téléchargement pour Windows.
2. Lancez le fichier téléchargé. Acceptez la licence.
3. Dans l'écran des options, cochez **« Ajouter l'action Ouvrir avec Code au menu contextuel »** pour les fichiers et les dossiers, ainsi que **« Ajouter à PATH »**. Ces options vous feront gagner beaucoup de temps.
4. Terminez l'installation et ouvrez VS Code.

Guide officiel détaillé : installer VS Code sur Windows.

Installer VS Code sur Mac

1. Téléchargez la version macOS sur code.visualstudio.com.
2. Ouvrez le fichier téléchargé : il contient l'application **Visual Studio Code**.
3. Glissez l'application dans le dossier **Applications**.
4. Ouvrez-la depuis le Launchpad ou le dossier Applications.

Guide officiel détaillé : installer VS Code sur macOS.

> **VOUS PREFEREZ APPRENDRE EN VIDEO ?**
>
> L'équipe de VS Code propose une série de courtes vidéos officielles qui montrent la prise en main de l'éditeur, l'ouverture d'un dossier et les premiers réglages : vidéos d'introduction à VS Code. Elles sont en anglais, mais très visuelles. Vous pouvez activer les sous-titres traduits automatiquement sur YouTube.

Deux extensions utiles

Une **extension** ajoute une fonction à VS Code. Pour les installer, cliquez sur l'icône des carrés dans la barre de gauche (ou `Ctrl+Shift+X` sur Windows, `Cmd+Shift+X` sur Mac), tapez le nom et cliquez sur *Install*.

- **French Language Pack for Visual Studio Code** (publié par Microsoft) : met l'interface en français. Facultatif, mais confortable au début.
- **Live Server** : ouvre votre page dans le navigateur et la **recharge automatiquement** à chaque enregistrement. Après l'installation, faites un clic droit sur `index.html` puis *Open with Live Server*.

> **N'INSTALLEZ PAS DES DIZAINES D'EXTENSIONS**
>
> Chaque extension a accès à vos fichiers et ralentit un peu l'éditeur. Installez uniquement ce dont vous avez besoin et préférez les extensions très téléchargées, publiées par des éditeurs connus. Une extension inconnue peut contenir du code malveillant.

1.5 Fichiers, dossiers et extensions

Beaucoup de débutants perdent du temps à cause de problèmes qui n'ont rien à voir avec le code : un fichier mal nommé, enregistré au mauvais endroit ou avec une mauvaise extension. Prenons cinq minutes pour éviter ces pièges.

L'extension est la fin du nom de fichier, après le point : `.html`, `.css`, `.js`, `.jpg`. Elle indique à l'ordinateur la nature du fichier. Un fichier nommé `index.html.txt` n'est **pas** une page Web, c'est un fichier texte, même s'il contient du HTML.

Par défaut, Windows cache souvent les extensions. Affichez-les une fois pour toutes :

- **Windows 11** : dans l'Explorateur de fichiers, menu **Afficher** → **Afficher** → cochez **Extensions de noms de fichiers**.
- **Windows 10** : onglet **Affichage** → cochez **Extensions de noms de fichiers**.
- **Mac** : dans le Finder, menu **Finder** → **Réglages** → **Avancé** → cochez **Afficher toutes les extensions de fichier**.

Adoptez aussi ces habitudes de nommage dès maintenant :

À FAIRE	À EVITER	POURQUOI
`mon-portfolio`	`Mon Portfolio (2)`	Les espaces et parenthèses posent problème dans les adresses Web
`photo-profil.jpg`	`Photo Profil.JPG`	Sur un serveur, majuscules et minuscules sont différentes
`a-propos.html`	`à propos.html`	Les accents peuvent mal s'afficher dans les liens

La règle simple : **minuscules, sans accent, sans espace, avec des tirets**.

1.6 Ouvrir un projet dans VS Code

Dans VS Code, on ouvre toujours un **dossier**, pas un fichier isolé. C'est le dossier qui représente votre projet.

1. Créez un dossier `premier-site` dans vos Documents.

2. Dans VS Code : **Fichier** → **Ouvrir le dossier…** et choisissez `premier-site`. Si VS Code vous demande si vous faites confiance aux auteurs des fichiers, répondez oui : c'est votre propre dossier.
3. Dans le panneau de gauche, appelé **Explorateur**, cliquez sur l'icône *Nouveau fichier* et nommez-le `index.html`.
4. Tapez votre code, puis enregistrez avec `Ctrl+S` (Windows) ou `Cmd+S` (Mac).
5. Ouvrez la page : double-cliquez sur le fichier dans l'explorateur de fichiers de votre ordinateur, ou utilisez Live Server.

> **LE RACCOURCI MAGIQUE**
>
> Dans un fichier `.html` vide, tapez un point d'exclamation `!` puis appuyez sur la touche `Tab`. VS Code génère automatiquement la structure de base d'une page HTML. Nous verrons au chapitre 2 ce que signifie chaque ligne. Ne l'utilisez qu'après l'avoir écrite au moins une fois à la main.

1.7 Votre meilleur allié : les outils de développement

Tous les navigateurs modernes contiennent des **outils de développement**, souvent appelés *DevTools*. Ils permettent de voir le code d'une page, de tester des modifications en direct et, surtout, de lire les **messages d'erreur**.

Pour les ouvrir sur ordinateur :

- **Windows** : touche `F12`, ou `Ctrl+Shift+I` ;
- **Mac** : `Cmd+Option+I` ;
- ou clic droit sur un élément de la page → **Inspecter**.

Vous verrez plusieurs onglets. Deux vont vous servir tout au long du livre :

- **Éléments** (*Elements*) : la structure HTML de la page et les styles CSS appliqués à chaque élément ;
- **Console** : les messages et les erreurs JavaScript. Quand « rien ne se passe », c'est ici qu'il faut regarder en premier.

> **À RETENIR**
>
> Un bon développeur n'est pas quelqu'un qui ne fait jamais d'erreur. C'est quelqu'un qui sait **où regarder** quand une erreur se produit. Les outils de développement sont ce premier endroit.

1.8 Premier exercice guidé

Faisons ensemble votre première vraie page, étape par étape. Suivez exactement ces instructions, sur téléphone ou sur ordinateur.

Étape 1. Dans votre dossier `premier-site`, créez le fichier `index.html`.

Étape 2. Tapez ce code à la main. Ne le copiez pas : tapez-le, même si cela prend cinq minutes.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Ma première page</title>
</head>
<body>
  <h1>Bonjour, je commence le développement Web !</h1>
  <p>Je m'appelle [votre prénom] et j'apprends à créer des sites.</p>
</body>
</html>
```

Étape 3. Remplacez `[votre prénom]` par votre prénom, puis enregistrez.

Étape 4. Ouvrez la page dans le navigateur. Vous devez voir un grand titre en gras et une phrase en dessous. L'onglet du navigateur doit afficher *Ma première page*.

Étape 5. Faites une erreur volontaire : supprimez le `>` à la fin de `<h1>`. Enregistrez et rechargez. Que se passe-t-il ? Remettez le caractère, rechargez. Vous venez de faire votre premier débogage.

Si quelque chose ne fonctionne pas, vérifiez dans l'ordre : le nom du fichier (`index.html`, pas `index.html.txt`), l'enregistrement (un point ou un rond sur l'onglet de VS Code signale un fichier non enregistré), puis l'orthographe des balises.

EXERCICES

1. Sur votre téléphone ou votre ordinateur, créez un fichier `index.html` qui affiche « Bonjour, je commence le développement Web ! ».
2. Ajoutez un deuxième titre avec `<h2>` et deux paragraphes avec `<p>`.
3. Sur ordinateur si vous en avez un : installez VS Code, l'extension Live Server, puis ouvrez votre dossier `premier-site`.
4. Activez l'affichage des extensions de fichiers sur votre ordinateur.
5. Ouvrez les outils de développement sur une page que vous visitez souvent et cliquez sur *Inspecter* sur un titre. Observez le code qui apparaît, sans chercher à tout comprendre.

UNE INSTALLATION QUI BLOQUE ?

Les installations sont souvent l'étape la plus frustrante, et la moins intéressante. Ne restez pas bloqué une journée entière : écrivez-moi sur WhatsApp en indiquant votre appareil, votre système (Windows, Mac, Android, iPhone) et une capture d'écran du message affiché.

PARTIE I

HTML

Construire la page : donner une structure et un sens au contenu.

CHAPITRE 02

Comprendre HTML

HTML ne sert pas à faire joli. Il sert à dire au navigateur ce qu'est chaque morceau de votre page.

HTML signifie **HyperText Markup Language**, que l'on peut traduire par « langage de balisage hypertexte ». Ce n'est pas un langage de programmation au sens classique : il ne calcule rien, il ne prend aucune décision. Il sert à **structurer le contenu** d'une page Web : ceci est un titre, ceci est un paragraphe, ceci est une image, ceci est un lien.

Si une page Web était une maison, HTML serait son **squelette et ses pièces** : les murs, les portes, la cuisine, le salon. CSS viendra ensuite peindre les murs et choisir les meubles. JavaScript installera l'électricité.

2.1 Une balise, c'est quoi ?

Une **balise** est un mot placé entre deux chevrons `<` et `>`. Elle indique au navigateur la nature d'un contenu.

```
<h1>Mon premier site</h1>
<p>Bienvenue sur ma page.</p>
```

Décomposons la première ligne :

- `<h1>` est la **balise ouvrante** : « ici commence un titre de niveau 1 » ;
- `Mon premier site` est le **contenu** ;
- `</h1>` est la **balise fermante** : « ici se termine le titre ». Elle contient une barre oblique `/`.

L'ensemble, balise ouvrante + contenu + balise fermante, s'appelle un **élément**.

> **L'OUBLI LE PLUS FREQUENT**
>
> Oublier la balise fermante, ou oublier la barre `/` dans la balise fermante. Écrire `<p>Bonjour<p>` au lieu de `<p>Bonjour</p>` crée deux paragraphes au lieu d'un. Le navigateur essaie souvent de « réparer » vos erreurs, ce qui rend le résultat imprévisible.

Quelques balises n'ont pas de contenu et donc pas de balise fermante. On les appelle des **éléments vides**. Les plus courants sont `<img>` (image), `<br>` (retour à la ligne), `<input>` (champ de formulaire) et `<meta>` (information sur la page).

Les éléments s'emboîtent

Les éléments peuvent contenir d'autres éléments, comme des boîtes rangées dans des boîtes :

```
<p>J'apprends le <strong>HTML</strong> avec plaisir.</p>
```

Ici, l'élément `<strong>` (texte important, affiché en gras) est **à l'intérieur** du paragraphe. La règle d'or : **on ferme dans l'ordre inverse de l'ouverture**. La dernière boîte ouverte est la première fermée.

```
<!-- Correct -->
<p>Texte <strong>important</strong></p>

<!-- Incorrect : les balises se croisent -->
<p>Texte <strong>important</p></strong>
```

La ligne qui commence par `<!--` et finit par `-->` est un **commentaire**. Le navigateur l'ignore. Les commentaires servent à laisser des notes aux humains qui liront le code, y compris vous dans trois mois.

2.2 Le document HTML minimal

Une vraie page HTML possède toujours la même structure de base. Vous l'avez déjà tapée au chapitre 1. Regardons-la maintenant ligne par ligne.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mon site</title>
</head>
<body>
  <h1>Bonjour !</h1>
  <p>Je découvre HTML.</p>
</body>
</html>
```

LIGNE	CE QU'ELLE SIGNIFIE
`<!DOCTYPE html>`	« Ce document est une page HTML moderne. » Toujours en première ligne.
`<html lang="fr">`	Le début de la page. `lang="fr"` indique qu'elle est écrite en français.
`<head>`	La « tête » : des informations **sur** la page, invisibles dans la page elle-même.
`<meta charset="UTF-8">`	L'encodage des caractères. Sans lui, les accents (é, à, ç) peuvent s'afficher en symboles étranges.
`<meta name="viewport" …>`	Demande aux téléphones d'afficher la page à la bonne taille. Indispensable pour le mobile.
`<title>`	Le texte affiché dans l'onglet du navigateur et dans les résultats de Google.
`<body>`	Le « corps » : **tout ce que l'utilisateur voit**.

Ne cherchez pas à mémoriser chaque ligne dès le premier jour. Retenez surtout l'idée principale :

> **À RETENIR**
>
> `<head>` contient des **informations sur la page**. `<body>` contient **ce que l'utilisateur voit**. Tout votre contenu visible va dans `<body>`.

L'indentation : décaler pour mieux lire

Vous avez remarqué que certaines lignes sont décalées vers la droite avec des espaces. C'est l'**indentation**. Le navigateur l'ignore complètement, mais elle est essentielle pour les humains : elle montre d'un coup d'œil quel élément est à l'intérieur de quel autre. Utilisez deux espaces (ou la touche `Tab`) à chaque niveau d'emboîtement. Dans VS Code, le raccourci `Shift+Alt+F` (Windows) ou `Shift+Option+F` (Mac) réindente tout le fichier automatiquement.

2.3 Les balises essentielles

Il existe plus d'une centaine de balises HTML. Rassurez-vous : une quinzaine suffisent pour construire la plupart des pages. Les voici, rangées par famille.

Pour le texte

BALISE	ROLE	EXEMPLE
`h1` à `h6`	Titres, du plus important (`h1`) au moins important (`h6`)	`<h2>Mes projets</h2>`
`P`	Paragraphe	`<p>Bonjour.</p>`
`strong`	Texte important (gras)	`<strong>Attention</strong>`
`Em`	Texte mis en valeur (italique)	`<em>vraiment</em>`
`Br`	Retour à la ligne, sans balise fermante	`Ligne 1 Ligne 2`

> **UN SEUL H1 PAR PAGE**
>
> Le `h1` est le titre principal de la page, comme le titre d'un livre. Il n'y en a qu'un. Les sections utilisent `h2`, leurs sous-parties `h3`, et ainsi de suite. Ne choisissez jamais un niveau de titre pour sa taille : la taille se règle avec CSS.

Pour les listes

BALISE	ROLE	EXEMPLE
`Ul`	Liste à puces (non ordonnée)	`<ul>…</ul>`
`Ol`	Liste numérotée (ordonnée)	`<ol>…</ol>`
`li`	Un élément de liste, à l'intérieur de `ul` ou `ol`	`<li>HTML</li>`

```
<h2>Ce que j'apprends</h2>
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>
```

Pour les liens et les images

BALISE	ROLE	EXEMPLE
`a`	Lien vers une autre page ou un autre site	`<a href="contact.html">Contact</a>`
`img`	Image, sans balise fermante	`<img src="photo.jpg" alt="Mon portrait">`

Pour regrouper et interagir

BALISE	ROLE	EXEMPLE
`div`	Conteneur générique, sans signification particulière	`<div>…</div>`
`button`	Bouton cliquable	`<button>Envoyer</button>`
`form`	Formulaire	`<form>…</form>`

C'est tout pour l'instant. Avec ces quinze balises, vous pouvez déjà construire une page complète. Les autres, vous les découvrirez au fil des projets, au moment où vous en aurez besoin.

2.4 Les attributs

Un **attribut** apporte une information supplémentaire à une balise. Il s'écrit toujours dans la balise ouvrante, sous la forme `nom="valeur"`.

```
<a href="https://developer.mozilla.org/fr/">Documentation MDN</a>
```

Ici, `href` est l'attribut, et l'adresse entre guillemets est sa valeur. Elle indique **où mène le lien**. Sans `href`, le lien ne mène nulle part.

Pour une image, deux attributs sont indispensables :

```
<img src="images/portrait.jpg" alt="David souriant devant son ordinateur">
```

- `src` (*source*) indique le fichier image à afficher ;
- `alt` (*alternative*) décrit l'image en mots.

Le texte `alt` est affiché si l'image ne se charge pas. Il est lu à voix haute par les lecteurs d'écran qu'utilisent les personnes aveugles ou malvoyantes, et il aide les moteurs de recherche à comprendre l'image. Décrivez ce que l'image **montre**, en une phrase courte. N'écrivez pas « image de… » : le lecteur d'écran l'annonce déjà.

D'autres attributs reviendront souvent :

ATTRIBUT	SUR QUELLES BALISES	À QUOI IL SERT
`href`	`A`	Destination du lien
`src`	`img`, `script`	Fichier à charger
`alt`	`Img`	Description de l'image
`id`	toutes	Nom **unique** d'un élément dans la page
`class`	toutes	Nom de groupe, réutilisable, pour le style (chapitre 5)

ATTRIBUT	SUR QUELLES BALISES	À QUOI IL SERT
`lang`	`Html`	Langue de la page

EXERCICES

1. Créez une page avec un titre `h1`, deux sous-titres `h2` et trois paragraphes.
2. Ajoutez une liste à puces de cinq objectifs personnels pour cette année.
3. Ajoutez une liste numérotée des trois étapes de votre journée idéale.
4. Ajoutez un lien vers un site que vous consultez régulièrement.
5. Ajoutez une image (placez un fichier `.jpg` dans votre dossier) avec un texte `alt` pertinent.
6. Faites une erreur volontaire : croisez deux balises comme `<p><strong>…</p></strong>`. Ouvrez les outils de développement, onglet *Éléments* : comment le navigateur a-t-il interprété votre code ?

CHAPITRE 03

Organiser une vraie page HTML

Quand votre page grandit, la structure devient plus importante que les détails.

Une page d'une vingtaine de lignes reste lisible même si elle est désordonnée. Une page de trois cents lignes, non. Il est temps d'apprendre à organiser votre contenu comme le font les professionnels.

3.1 Les éléments sémantiques

Pendant longtemps, les développeurs construisaient leurs pages uniquement avec des `<div>`. Le problème : `<div>` ne veut rien dire. En lisant `<div>`, ni le navigateur, ni Google, ni un lecteur d'écran ne savent s'il s'agit d'un menu, d'un article ou d'un pied de page.

HTML propose des éléments dits **sémantiques**, c'est-à-dire qui portent un **sens**. Ils fonctionnent comme des `<div>`, mais ils annoncent clairement leur rôle.

```
<body>
  <header>
    <h1>David Mbula</h1>
    <nav>Navigation</nav>
  </header>

  <main>
    <section>
      <h2>À propos</h2>
      <p>…</p>
    </section>
    <section>
      <h2>Mes projets</h2>
      <p>…</p>
    </section>
  </main>

  <footer>
    <p>© 2026 David Mbula</p>
  </footer>
</body>
```

ÉLEMENT	SON ROLE
`header`	L'en-tête : logo, nom du site, souvent la navigation
`nav`	Le menu de navigation principal
`main`	Le contenu principal, unique dans la page
`section`	Une partie thématique, avec en général son propre titre `h2`
`article`	Un contenu autonome : un article de blog, une fiche projet
`footer`	Le pied de page : contact, mentions, copyright

Visuellement, ces éléments ne changent rien : sans CSS, un `<header>` ressemble exactement à un `<div>`. Mais votre code devient **lisible pour les humains et compréhensible pour les machines**. Votre page est mieux référencée, plus accessible et plus facile à maintenir.

> **À RETENIR**
>
> Avant d'écrire un `<div>`, demandez-vous : « Existe-t-il un élément qui décrit mieux ce contenu ? » Si oui, utilisez-le. Gardez `<div>` pour les regroupements purement visuels.

3.2 La navigation et les liens

Un site comporte généralement plusieurs pages, reliées par un menu.

```
<nav>
  <a href="index.html">Accueil</a>
  <a href="a-propos.html">À propos</a>
  <a href="contact.html">Contact</a>
</nav>
```

Liens relatifs et liens absolus

Il existe deux façons d'écrire l'adresse d'un lien :

- un **lien absolu** contient l'adresse complète, avec `https://`. Il sert à pointer vers un **autre site** : `href="https://github.com"` ;
- un **lien relatif** indique un chemin **à partir du fichier actuel**. Il sert à pointer vers une page **de votre site** : `href="contact.html"`.

Pour les chemins relatifs, retenez trois cas :

CHEMIN	SIGNIFICATION
`contact.html`	Le fichier est dans le **même dossier**
`pages/contact.html`	Le fichier est dans le sous-dossier `pages`
`../index.html`	Le fichier est dans le **dossier parent** (on remonte d'un niveau)

Les liens internes à la page

Un lien peut aussi mener à une section de la même page. Donnez un `id` à la section, puis utilisez `#` suivi de cet `id` :

```
<a href="#projets">Voir mes projets</a>

<section id="projets">
  <h2>Mes projets</h2>
</section>
```

Ouvrir dans un nouvel onglet

Pour les liens vers d'autres sites, il est courant d'ouvrir un nouvel onglet :

```
<a href="https://github.com" target="_blank" rel="noopener">Mon GitHub</a>
```

L'attribut `rel="noopener"` est une bonne pratique de sécurité : il empêche la page ouverte d'agir sur la vôtre. N'en abusez pas : ouvrir chaque lien dans un nouvel onglet finit par agacer les visiteurs.

3.3 Les images en pratique

Organisez vos images dans un dossier dédié :

```
mon-portfolio/
├── index.html
└── images/
    ├── portrait.jpg
    └── projet-1.jpg
```

```
<img src="images/portrait.jpg" alt="Portrait de David" width="400" height="400">
```

Les attributs `width` et `height` indiquent la taille de l'image en pixels. Le navigateur réserve ainsi la bonne place avant même que l'image soit chargée, ce qui évite que la page « saute » pendant le chargement.

> **LES IMAGES TROP LOURDES**
>
> Une photo prise avec un téléphone pèse souvent 3 à 5 Mo. Sur une connexion mobile lente, elle peut mettre de longues secondes à s'afficher et consommer le forfait de vos visiteurs. Pour le Web, redimensionnez vos photos (1 200 pixels de large suffisent largement) et compressez-les avec un outil gratuit comme Squoosh, directement dans le navigateur. Visez moins de 300 Ko par image.

3.4 Les formulaires

Un formulaire permet de recueillir des informations : un nom, un e-mail, un message.

```
<form>
  <label for="nom">Votre nom</label>
  <input id="nom" name="nom" type="text" required>

  <label for="email">Votre e-mail</label>
  <input id="email" name="email" type="email" required>

  <label for="message">Votre message</label>
  <textarea id="message" name="message" rows="5"></textarea>

  <button type="submit">Envoyer</button>
</form>
```

Les éléments clés :

- `<label>` est l'**étiquette** du champ. Son attribut `for` doit correspondre à l'`id` du champ. Grâce à ce lien, cliquer sur l'étiquette place le curseur dans le champ, et les lecteurs d'écran annoncent correctement chaque champ ;
- `<input>` est un champ d'une ligne. Son attribut `type` change son comportement : `text`, `email`, `password`, `number`, `date`, `checkbox`… Sur téléphone, `type="email"` affiche même un clavier adapté, avec le `@` ;
- `<textarea>` est un champ de plusieurs lignes, pour un message ;
- `name` est le nom sous lequel la donnée sera envoyée ;
- `required` rend le champ obligatoire : le navigateur refuse l'envoi s'il est vide.

Pour l'instant, ce formulaire ne s'envoie nulle part : il faut un serveur pour recevoir les données. Au chapitre 9, JavaScript réagira à sa soumission. Vous découvrirez le traitement côté serveur dans le tome 2 de la série.

LE PLACEHOLDER NE REMPLACE PAS LE LABEL

L'attribut `placeholder` affiche un texte gris dans le champ vide (« ex. : jean@mail.com »). Il disparaît dès que l'on tape, et il est souvent mal lu par les lecteurs d'écran. Utilisez-le comme exemple, jamais à la place d'un `<label>`.

3.5 Valider votre HTML

Comment savoir si votre code est correct ? Le W3C, l'organisme qui définit les standards du Web, propose un **validateur gratuit** : validator.w3.org. Collez votre code dans l'onglet *Validate by Direct Input* et cliquez sur *Check*. Le validateur liste les erreurs, avec le numéro de ligne.

Les messages sont en anglais, mais souvent très clairs : *Unclosed element "p"* signifie qu'un paragraphe n'a pas été fermé.

AVEC L'IA

Un message d'erreur du validateur vous semble incompréhensible ? C'est un excellent usage de l'IA. Copiez **le message et la ligne concernée**, puis demandez : « Explique-moi ce message d'erreur HTML avec des mots simples, et dis-moi où regarder dans mon code, **sans me donner la correction** ». Vous comprendrez l'erreur et la corrigerez vous-même. C'est exactement ce qui vous rendra autonome.

EXERCICES

1. Construisez une page portfolio avec `header`, `nav`, `main`, au moins deux `section` et un `footer`.
2. Ajoutez dans la navigation des liens internes vers chacune de vos sections, grâce aux `id`.
3. Créez un formulaire de contact avec nom, e-mail et message.
4. Vérifiez que chaque champ possède un `label` correctement relié par `for` et `id`.
5. Passez votre page dans le validateur du W3C et corrigez toutes les erreurs.
6. Expliquez à voix haute, comme à un ami, le rôle de `header`, `nav`, `main`, `section` et `footer`.

CHAPITRE 04

Mini-projet HTML : votre page de présentation

Il est temps d'assembler tout ce que vous avez appris dans un vrai projet : la première version de votre portfolio.

Vous allez créer une page « Présentation personnelle ». C'est le projet qui vous accompagnera jusqu'à la fin du livre : vous l'embellirez avec CSS, l'animerez avec JavaScript, le sauvegarderez avec Git et le publierez avec GitHub.

4.1 Le cahier des charges

Votre page doit contenir :

- votre nom et une phrase qui vous résume ;
- une courte présentation (qui vous êtes, ce que vous apprenez, ce que vous visez) ;
- une liste de vos compétences, même débutantes ;
- au moins un projet, même modeste (cette page elle-même compte) ;
- une image : votre portrait, ou une illustration si vous préférez ;
- au moins trois liens ;
- un formulaire de contact.

Travaillez sans CSS. Le résultat sera très simple, en noir sur blanc, avec la police par défaut du navigateur. C'est volontaire. Une page bien structurée en HTML pur est déjà lisible et utilisable. Le design viendra ensuite, et il sera beaucoup plus facile à réaliser sur une structure propre.

4.2 Avant de coder : faites un plan

Les développeurs professionnels ne commencent presque jamais par le code. Ils commencent par un plan. Prenez une feuille et dessinez votre page en rectangles : où est le titre, où sont les sections, dans quel ordre. Puis transformez ce dessin en liste :

```
header : nom + phrase d'accroche + navigation
main
  section #a-propos : titre + 2 paragraphes + portrait
  section #competences : titre + liste
  section #projets : titre + 1 article par projet
  section #contact : titre + formulaire
footer : copyright + liens
```

Ce plan de dix lignes vaut de l'or : il vous évite de vous perdre, et il se traduit presque directement en HTML. C'est la première application concrète de la philosophie de cette série : **penser avant de générer**, et même avant d'écrire.

4.3 Une structure de départ

Voici un squelette possible. Tapez-le, puis remplacez chaque contenu par le vôtre.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Prénom Nom — Développeur Web débutant</title>
</head>
<body>
  <header>
    <h1>Prénom Nom</h1>
    <p>J'apprends le développement Web, une ligne à la fois.</p>
    <nav>
      <a href="#a-propos">À propos</a>
      <a href="#competences">Compétences</a>
      <a href="#projets">Projets</a>
      <a href="#contact">Contact</a>
    </nav>
  </header>

  <main>
    <section id="a-propos">
      <h2>À propos</h2>
      <img src="images/portrait.jpg" alt="Portrait de Prénom" width="300"
height="300">
      <p>Je m'appelle Prénom, j'habite à …</p>
      <p>Mon objectif : …</p>
    </section>

    <section id="competences">
      <h2>Compétences</h2>
      <ul>
        <li>HTML : structure d'une page, formulaires</li>
        <li>CSS : en cours d'apprentissage</li>
        <li>JavaScript : bientôt</li>
      </ul>
```

```
    </section>

    <section id="projets">
      <h2>Projets</h2>
      <article>
        <h3>Mon portfolio</h3>
        <p>La page que vous lisez, construite à la main en HTML.</p>
      </article>
    </section>

    <section id="contact">
      <h2>Contact</h2>
      <form>
        <label for="nom">Nom</label>
        <input id="nom" name="nom" type="text" required>
        <label for="email">E-mail</label>
        <input id="email" name="email" type="email" required>
        <label for="message">Message</label>
        <textarea id="message" name="message" rows="5"></textarea>
        <button type="submit">Envoyer</button>
      </form>
    </section>
  </main>

  <footer>
    <p>© 2026 Prénom Nom</p>
  </footer>
</body>
</html>
```

4.4 Les erreurs les plus fréquentes

Avant de considérer votre page comme terminée, passez-la au crible de cette liste. Ces erreurs reviennent chez presque tous les débutants.

SYMPTOME	CAUSE PROBABLE	SOLUTION
L'image ne s'affiche pas	Chemin ou nom de fichier incorrect, majuscule oubliée	Vérifiez que `src` correspond **exactement** au nom et à l'emplacement du fichier
Les accents s'affichent mal (Ã©)	`<meta charset="UTF-8">` absent	Ajoutez-le dans `<head>`
Tout le texte est en gras	Une balise `<strong>` ou `<h1>` non fermée	Cherchez la balise ouvrante sans sa fermante
La page ne change pas	Fichier non enregistré, ou mauvais fichier ouvert	Enregistrez (`Ctrl+S`) et vérifiez le nom de l'onglet

SYMPTOME	CAUSE PROBABLE	SOLUTION
Un lien interne ne fonctionne pas	L'`id` et le `#` ne correspondent pas	`href="#projets"` exige `id="projets"`, sans `#` dans l'`id`

À RETENIR

Quand quelque chose ne marche pas, procédez comme un enquêteur : **une hypothèse à la fois**. Vérifiez l'enregistrement, puis le nom du fichier, puis l'orthographe, puis la fermeture des balises. Changer cinq choses en même temps vous empêche de savoir laquelle a résolu le problème.

EXERCICES

1. Dessinez le plan de votre page sur papier, puis écrivez-le sous forme de liste.
2. Créez votre `index.html` en suivant votre plan.
3. Organisez la page avec des éléments sémantiques.
4. Ajoutez au moins trois liens, une image avec un vrai texte `alt` et le formulaire de contact.
5. Testez la page sur téléphone et sur ordinateur si possible.
6. Passez-la dans le validateur du W3C et corrigez les balises mal fermées ou mal imbriquées.
7. Gardez une copie de ce fichier dans un dossier `version-html`. À la fin du chapitre 7, vous comparerez l'avant et l'après.

II

PARTIE II

CSS

Transformer la structure en design : couleurs, espaces, mise en page et adaptation au mobile.

CHAPITRE 05

Comprendre CSS

HTML dit ce qu'est le contenu. CSS dit à quoi il ressemble.

CSS signifie **Cascading Style Sheets**, « feuilles de style en cascade ». Il sert à contrôler l'apparence de votre HTML : couleurs, polices, tailles, espacements, bordures, alignements et disposition des éléments sur l'écran.

La séparation est essentielle : **HTML pour la structure, CSS pour l'apparence**. Grâce à elle, vous pouvez changer complètement le design d'un site sans toucher à son contenu, ou réutiliser le même style sur des dizaines de pages.

5.1 Relier CSS à HTML

Le CSS s'écrit dans un fichier séparé, avec l'extension `.css`. Créez un fichier `style.css` à côté de votre `index.html`, puis reliez-le dans le `<head>` :

```
<head>
  <meta charset="UTF-8">
  <title>Mon portfolio</title>
  <link rel="stylesheet" href="style.css">
</head>
```

Le navigateur charge alors `style.css` et applique ses règles à la page. Pour vérifier que le lien fonctionne, écrivez une règle très visible dans `style.css` :

```
body {
  background-color: yellow;
}
```

Si la page devient jaune, le lien fonctionne. Si rien ne change, vérifiez que le nom du fichier dans `href` correspond exactement au nom réel, et que les deux fichiers sont dans le même dossier. Ensuite, supprimez cette règle : le jaune n'était qu'un test.

> **IL EXISTE D'AUTRES FAÇONS D'ECRIRE DU CSS**
>
> Vous verrez parfois du CSS directement dans une balise (`<p style="color: red">`) ou dans une balise `<style>` dans le `<head>`. Ces méthodes fonctionnent, mais elles mélangent structure et apparence. Pour vos projets, gardez la bonne habitude : **un fichier CSS séparé**.

5.2 Anatomie d'une règle CSS

```
h1 {
  color: navy;
  font-size: 32px;
}

p {
  line-height: 1.6;
}
```

Une **règle CSS** se compose de trois parties :

- le **sélecteur** (`h1`, `p`) : **quels** éléments sont concernés ;
- la **propriété** (`color`, `font-size`) : **ce que** l'on modifie ;
- la **valeur** (`navy`, `32px`) : **comment** on le modifie.

Les déclarations sont placées entre accolades `{ }`. Chaque déclaration se termine par un point-virgule `;`. Oublier un point-virgule est l'erreur CSS la plus fréquente : la déclaration suivante est alors ignorée, sans aucun message d'erreur.

Traduisons la première règle en français : « Pour tous les titres `h1`, mets le texte en bleu marine et la taille à 32 pixels. »

5.3 Les sélecteurs essentiels

SELECTEUR	EXEMPLE	CE QU'IL CIBLE
Balise	`p`	Tous les paragraphes
Classe	`.important`	Tous les éléments qui ont `class="important"`
Identifiant	`#contact`	L'unique élément qui a `id="contact"`
Descendant	`nav a`	Les liens situés **à l'intérieur** d'un `nav`
Groupe	`h1, h2`	Les `h1` **et** les `h2`
État	`a:hover`	Un lien **survolé** par la souris

Les classes, votre outil principal

Une **classe** est un nom que vous inventez et que vous attribuez à un ou plusieurs éléments HTML :

```
<p class="important">Ce texte est important.</p>
<p>Ce texte est normal.</p>
<li class="important">Cet élément aussi est important.</li>
```

```
.important {
  font-weight: bold;
  color: #5d1212;
}
```

Notez le **point** devant le nom de la classe dans le CSS, et son absence dans le HTML. Un élément peut avoir plusieurs classes, séparées par un espace : `class="carte mise-en-avant"`.

> **À RETENIR**
>
> Utilisez **les classes** pour styliser. Gardez les `id` pour les liens internes et pour JavaScript. Une classe se réutilise à volonté ; un `id` est unique dans la page.

Les états : survol et focus

```
a:hover {
  text-decoration: underline;
}

button:focus-visible {
  outline: 3px solid orange;
}
```

`:hover` s'applique quand la souris survole l'élément. `:focus-visible` s'applique quand l'élément est sélectionné au clavier, avec la touche `Tab`. Ne supprimez jamais complètement le contour de focus : les personnes qui naviguent au clavier en ont besoin pour savoir où elles se trouvent.

5.4 La cascade : qui gagne ?

Que se passe-t-il quand deux règles se contredisent ?

```
p { color: black; }
.important { color: red; }
p { color: blue; }
```

Le navigateur suit deux principes, que l'on appelle la **cascade** :

1. **La règle la plus précise gagne.** Un `id` est plus précis qu'une classe, qui est plus précise qu'une balise. Un paragraphe avec `class="important"` sera donc rouge, même si une règle `p` arrive après.

2. **À précision égale, la dernière règle écrite gagne.** Les paragraphes normaux seront bleus, car `p { color: blue; }` arrive après `p { color: black; }`.

Quand un style « ne s'applique pas », c'est presque toujours parce qu'une autre règle, plus précise ou écrite plus bas, l'emporte. Les outils de développement vous le montrent : dans l'onglet *Éléments*, les déclarations barrées sont celles qui ont perdu.

> **LE PIEGE DU ! IMPORTANT**
>
> Vous croiserez `color: red !important;`. Ce mot force une règle à gagner contre toutes les autres. C'est tentant, mais c'est un pansement qui crée de nouveaux problèmes : pour annuler un `!important`, il faut un autre `!important`. Cherchez plutôt pourquoi votre règle perd.

5.5 Couleurs, unités et typographie

Les couleurs

NOTATION	EXEMPLE	REMARQUE
Nom	`red`, `navy`, `white`	Simple, mais choix limité
Hexadécimal	`#5d1212`	La plus utilisée. Trois paires : rouge, vert, bleu
RGB	`rgb(93, 18, 18)`	Même principe, en nombres de 0 à 255
RGB + transparence	`rgb(0 0 0 / 50%)`	Le dernier nombre règle l'opacité

Pour choisir des couleurs harmonieuses, limitez-vous au début à **trois couleurs** : une couleur de fond claire, une couleur de texte foncée et une couleur d'accent pour les liens et les boutons. Vérifiez que le texte reste lisible : un gris clair sur fond blanc fatigue les yeux et exclut les personnes qui voient mal.

Les unités de taille

UNITE	SIGNIFICATION	USAGE CONSEILLE
`Px`	Pixel	Bordures, petits détails
`rem`	Relative à la taille de texte de base (souvent 16 px)	Tailles de texte, espacements
`%`	Pourcentage de l'élément parent	Largeurs
`vh` / `vw`	Pourcentage de la hauteur / largeur de l'écran	Sections plein écran

Préférez `rem` pour les textes : si un utilisateur augmente la taille du texte dans son navigateur, votre page s'adapte.

La typographie

```
body {
  font-family: system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
  font-size: 1rem;
  line-height: 1.6;
  color: #1c1c1a;
}

h1 {
  font-size: 2.5rem;
  line-height: 1.1;
}
```

- `font-family` liste des polices **par ordre de préférence**. Le navigateur utilise la première disponible. `system-ui` utilise la police native de l'appareil : rapide et toujours lisible ;
- `line-height` règle l'interligne. Entre `1.5` et `1.7` pour un texte courant : c'est l'un des réglages qui améliorent le plus la lisibilité ;
- `font-weight` règle la graisse : `400` normal, `700` gras.

EXERCICES

1. Créez `style.css`, reliez-le à votre portfolio et vérifiez le lien avec un fond de couleur temporaire.
2. Changez la police, la taille et l'interligne du texte de la page.
3. Changez la taille et la couleur du titre principal.
4. Créez une classe `.important` et appliquez-la à deux éléments différents.
5. Ajoutez un effet de survol sur les liens de la navigation.
6. Écrivez deux règles contradictoires et prédisez laquelle va gagner **avant** de recharger la page. Vérifiez dans les outils de développement.

CHAPITRE 06

Le box model et la mise en page

En CSS, tout est une boîte. Comprendre ces boîtes, c'est comprendre la mise en page.

6.1 Le box model

Ouvrez les outils de développement, onglet *Éléments*, et cliquez sur n'importe quel élément. Dans le panneau des styles, vous verrez un schéma de rectangles emboîtés. C'est le **box model**, le modèle de boîte.

Chaque élément est une boîte composée de quatre couches, de l'intérieur vers l'extérieur :

1. le **contenu** (*content*) : le texte ou l'image ;
2. le **padding** : l'espace **intérieur**, entre le contenu et la bordure ;
3. la **bordure** (*border*) : le trait autour de la boîte ;
4. la **marge** (*margin*) : l'espace **extérieur**, entre la boîte et ses voisines.

```
.carte {
  width: 300px;
  padding: 20px;
  border: 1px solid #deddd6;
  margin: 20px;
}
```

Une image simple : la carte est un cadre photo. Le padding est le passe-partout blanc autour de la photo, la bordure est le cadre, la marge est l'espace entre ce cadre et le suivant sur le mur.

Le réglage à faire une fois pour toutes

Par défaut, `width: 300px` ne concerne que le contenu : la carte ci-dessus mesure en réalité 300 + 20 + 20 + 1 + 1 = **342 pixels** de large. Ce calcul surprend tous les débutants. Ajoutez cette règle **en haut de chaque fichier CSS** :

```
*, *::before, *::after {
  box-sizing: border-box;
}
```

Avec `border-box`, la largeur inclut le padding et la bordure. Une carte de 300 pixels mesure 300 pixels. Le sélecteur `*` signifie « tous les éléments ».

Raccourcis pour les espacements

```
padding: 20px;              /* les 4 côtés */
padding: 10px 20px;         /* haut-bas | gauche-droite */
padding: 10px 20px 30px 40px; /* haut | droite | bas | gauche */
margin: 0 auto;             /* centre horizontalement un bloc qui a une largeur */
```

Pour retenir l'ordre des quatre valeurs, pensez aux aiguilles d'une montre : on part du haut et on tourne dans le sens horaire.

6.2 Blocs et éléments en ligne

Les éléments HTML ne se placent pas tous de la même manière :

- les éléments **de bloc** (`h1`, `p`, `section`, `div`…) prennent toute la largeur disponible et commencent sur une nouvelle ligne ;
- les éléments **en ligne** (`a`, `strong`, `em`, `img`…) se placent dans le flux du texte, les uns à côté des autres.

La propriété `display` permet de changer ce comportement. Deux de ses valeurs vont transformer vos mises en page : `flex` et `grid`.

6.3 Flexbox : aligner et distribuer

Flexbox est l'outil le plus utile pour aligner des éléments sur une ligne ou une colonne. On l'applique au **parent**, et ce sont ses **enfants directs** qui s'organisent.

```
<nav class="menu">
  <a href="#a-propos">À propos</a>
  <a href="#projets">Projets</a>
  <a href="#contact">Contact</a>
</nav>
```

```
.menu {
  display: flex;
  gap: 20px;
  justify-content: center;
  align-items: center;
}
```

PROPRIETE	ROLE	VALEURS COURANTES
`display: flex`	Active Flexbox sur le parent	—

PROPRIETE	ROLE	VALEURS COURANTES
`flex-direction`	Sens de l'alignement	`row` (ligne, par défaut), `column`
`gap`	Espace entre les enfants	`20px`, `1rem`
`justify-content`	Répartition sur l'axe principal	`flex-start`, `center`, `space-between`
`align-items`	Alignement sur l'axe secondaire	`stretch`, `center`, `flex-start`
`flex-wrap`	Passage à la ligne si la place manque	`nowrap`, `wrap`

Une rangée de cartes

```
.cartes {
  display: flex;
  flex-wrap: wrap;
  gap: 1.5rem;
}

.carte {
  flex: 1 1 250px;
}
```

La déclaration `flex: 1 1 250px` signifie : « chaque carte part d'une largeur de 250 pixels, peut grandir pour occuper l'espace libre, et peut rétrécir si nécessaire ». Combinée à `flex-wrap: wrap`, elle crée une grille qui passe automatiquement de trois colonnes sur ordinateur à une seule sur téléphone.

> **UN JEU POUR MAITRISER FLEXBOX**
>
> Flexbox Froggyest un petit jeu gratuit, disponible en français, où vous placez des grenouilles sur des nénuphars en écrivant du CSS Flexbox. Une demi-heure de jeu vaut des heures de lecture.

6.4 Le responsive design

Une page **responsive** s'adapte à toutes les tailles d'écran. Un site qui paraît parfait sur un ordinateur mais déborde sur un téléphone **n'est pas terminé**. Dans beaucoup de pays, la majorité des visiteurs naviguent sur mobile.

Trois ingrédients rendent une page responsive :

1. la balise `<meta name="viewport">` dans le `<head>` (vous l'avez déjà) ;
2. des largeurs **flexibles** plutôt que fixes ;
3. des **media queries** pour ajuster le style selon la taille de l'écran.

Des largeurs flexibles

```
.container {
  max-width: 1100px;
  margin: 0 auto;
  padding: 0 1rem;
}

img {
  max-width: 100%;
  height: auto;
}
```

`max-width` signifie « au maximum ». Sur un grand écran, le contenu ne dépasse pas 1 100 pixels et reste centré. Sur un téléphone, il occupe toute la largeur disponible. La règle sur `img` empêche les images de déborder de leur conteneur.

Les media queries

```
.menu {
  display: flex;
  gap: 20px;
}

@media (max-width: 600px) {
  .menu {
    flex-direction: column;
    gap: 10px;
  }
}
```

Une media query applique des règles **seulement si une condition est remplie**. Ici : « si l'écran fait 600 pixels de large ou moins, empile les liens du menu verticalement ».

Tester sur mobile sans téléphone

Dans les outils de développement, cliquez sur l'icône représentant un téléphone et une tablette (ou `Ctrl+Shift+M`). Le navigateur simule différents écrans. Vous pouvez aussi simplement réduire la largeur de la fenêtre et observer comment la page réagit.

> **À RETENIR**
>
> Concevez d'abord pour le **petit écran**, puis ajoutez des règles pour les écrans plus grands. Une page simple sur mobile s'étend facilement ; une page complexe sur ordinateur se compresse difficilement.

EXERCICES

1. Ajoutez la règle `box-sizing: border-box` en haut de votre CSS.
2. Dans les outils de développement, observez le box model d'un de vos titres. Modifiez ses marges en direct.
3. Transformez votre liste de compétences en cartes alignées avec Flexbox.
4. Ajoutez un espace régulier entre les cartes avec `gap`.
5. Créez une classe `.container` qui limite la largeur de la page et la centre.
6. Ajoutez une media query pour les écrans de moins de 600 pixels.
7. Testez votre page en mode mobile dans les outils de développement, puis sur un vrai téléphone si possible.

CHAPITRE 07

Mini-projet CSS : un portfolio professionnel

Reprenez votre page HTML et donnez-lui une vraie identité visuelle.

7.1 Définir une identité avec les variables CSS

Avant d'écrire la moindre règle, choisissez vos couleurs et vos tailles. Les **variables CSS**, appelées *custom properties*, permettent de les définir une seule fois et de les réutiliser partout.

```
:root {
  --couleur-fond: #f5f5f2;
  --couleur-texte: #1c1c1a;
  --couleur-accent: #5d1212;
  --couleur-ligne: #deddd6;
  --rayon: 12px;
  --espace: 1.5rem;
}

body {
  background-color: var(--couleur-fond);
  color: var(--couleur-texte);
}

a {
  color: var(--couleur-accent);
}
```

`:root` représente la racine du document : les variables qui y sont définies sont disponibles partout. Une variable commence toujours par deux tirets, et on la lit avec `var()`. Le jour où vous voulez changer la couleur d'accent, vous modifiez **une seule ligne**. Au chapitre 10, ces variables vous permettront aussi de créer un thème sombre en quelques lignes.

7.2 Une feuille de style complète, section par section

Voici une base solide pour votre portfolio. Ne la copiez pas d'un bloc : écrivez-la section par section, en rechargeant la page après chacune pour voir son effet.

```
/* 1. Réglages de base */
```

```
*, *::before, *::after { box-sizing: border-box; }

body {
  margin: 0;
  font-family: system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
  line-height: 1.6;
  background-color: var(--couleur-fond);
  color: var(--couleur-texte);
}

img { max-width: 100%; height: auto; }

/* 2. Mise en page */
.container {
  max-width: 1000px;
  margin: 0 auto;
  padding: 0 1rem;
}

section {
  padding: 3rem 0;
  border-bottom: 1px solid var(--couleur-ligne);
}

/* 3. En-tête et navigation */
header {
  padding: 3rem 0 2rem;
}

header h1 {
  font-size: 2.5rem;
  line-height: 1.1;
  margin: 0;
}

.menu {
  display: flex;
  flex-wrap: wrap;
  gap: 1rem;
  margin-top: 1.5rem;
}

.menu a {
  color: var(--couleur-texte);
  text-decoration: none;
  font-weight: 600;
}

.menu a:hover {
  color: var(--couleur-accent);
}
```

```
/* 4. Cartes de projets et de compétences */
.cartes {
  display: flex;
  flex-wrap: wrap;
  gap: var(--espace);
}

.carte {
  flex: 1 1 250px;
  padding: var(--espace);
  background-color: white;
  border: 1px solid var(--couleur-ligne);
  border-radius: var(--rayon);
}

.carte h3 { margin-top: 0; }

/* 5. Boutons */
.bouton {
  display: inline-block;
  padding: 0.75rem 1.5rem;
  background-color: var(--couleur-accent);
  color: white;
  border: none;
  border-radius: var(--rayon);
  font: inherit;
  font-weight: 600;
  text-decoration: none;
  cursor: pointer;
}

.bouton:hover { opacity: 0.9; }

/* 6. Formulaire */
form {
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
  max-width: 500px;
}

input, textarea {
  padding: 0.75rem;
  border: 1px solid var(--couleur-ligne);
  border-radius: 8px;
  font: inherit;
}

/* 7. Petits écrans */
@media (max-width: 600px) {
  header h1 { font-size: 2rem; }
```

```
  section { padding: 2rem 0; }
}
```

Pour que ces styles s'appliquent, ajoutez les classes correspondantes dans votre HTML : `class="container"` sur un `div` qui entoure le contenu du `header`, du `main` et du `footer`, `class="menu"` sur le `nav`, `class="cartes"` sur le conteneur des projets, `class="carte"` sur chaque `article`, et `class="bouton"` sur le bouton du formulaire.

> **NE CHERCHEZ PAS LA PERFECTION**
>
> Votre premier design ne sera pas celui d'une grande agence, et c'est normal. Visez trois qualités : **lisible** (texte assez grand et contrasté), **aéré** (des espacements réguliers) et **cohérent** (les mêmes couleurs et les mêmes arrondis partout). Ces trois qualités suffisent à donner une impression professionnelle.

7.3 Observer, comparer, ajuster

Le design s'apprend en observant. Ouvrez deux ou trois portfolios de développeurs que vous trouvez réussis et posez-vous ces questions :

- Combien de couleurs utilisent-ils ? (Souvent deux ou trois, pas plus.)
- Quelle est la largeur maximale du texte ? (Rarement plus de 70 caractères par ligne.)
- Comment les sections sont-elles séparées ? (Par l'espace, plus que par des traits.)
- Qu'est-ce qui attire l'œil en premier ? (En général le nom et un bouton d'action.)

Les outils de développement vous permettent de voir le CSS de n'importe quel site. **Inspirez-vous des idées, pas du code** : comprendre pourquoi un design fonctionne vous apprendra bien plus que le recopier.

> **AVEC L'IA**
>
> Une fois votre version terminée, montrez votre CSS à un assistant IA et demandez : « Quelles sont les trois améliorations les plus importantes pour la lisibilité et la cohérence de ce design ? Explique pourquoi pour chacune. » Puis appliquez vous-même celles que vous comprenez et approuvez. Vous restez le designer ; l'IA est un relecteur.

EXERCICES

1. Définissez vos variables de couleurs et d'espacement dans `:root`.
2. Créez une classe `.container` et utilisez-la dans le `header`, le `main` et le `footer`.
3. Créez une classe `.carte` pour vos projets et vos compétences.
4. Stylisez vos liens et créez une classe `.bouton`.
5. Utilisez Flexbox pour au moins deux sections.
6. Ajoutez au moins une media query et testez sur mobile.
7. Comparez votre version HTML (chapitre 4) avec la version stylisée, et notez cinq changements qui ont le plus amélioré la page.

III

PARTIE III

JavaScript

Faire vivre la page : réagir aux clics, modifier le contenu, vérifier un formulaire.

CHAPITRE 08

Comprendre JavaScript

HTML structure, CSS décore. JavaScript agit.

JavaScript est un véritable **langage de programmation**. Contrairement à HTML et CSS, il peut prendre des décisions, répéter des actions, faire des calculs et réagir à ce que fait l'utilisateur. Il permet de donner un **comportement** à votre page : afficher un message après un clic, changer de thème, vérifier qu'un formulaire est correctement rempli, charger des données depuis Internet.

C'est aussi la partie qui demande le plus de patience. Ne vous inquiétez pas si les premières heures vous semblent difficiles : c'est le cas pour tout le monde. Avancez lentement, et tapez chaque exemple.

> **JAVA N'EST PAS JAVASCRIPT**
> Malgré leurs noms, Java et JavaScript sont deux langages complètement différents. La ressemblance des noms est un choix commercial des années 1990. Quand vous cherchez de l'aide en ligne, écrivez bien « JavaScript », ou son abréviation « JS ».

8.1 Relier JavaScript à votre page

Créez un fichier `script.js` à côté de vos autres fichiers, puis reliez-le juste avant la fermeture de `</body>` :

```
  <script src="script.js"></script>
</body>
```

Pourquoi à la fin ? Parce que le navigateur lit la page de haut en bas. Si le script est chargé avant le HTML, il cherchera des éléments qui n'existent pas encore. En le plaçant à la fin, vous êtes sûr que toute la page est déjà construite quand il s'exécute.

8.2 La console, votre carnet de bord

La **console** est l'endroit où JavaScript écrit ses messages et ses erreurs. Ouvrez les outils de développement (`F12`), onglet *Console*.

Écrivez dans `script.js` :

```
console.log("Le script est bien chargé !");
```

Rechargez la page : le message apparaît dans la console. `console.log()` est l'outil de débogage le plus simple et le plus utilisé au monde. Quand vous ne savez pas ce que contient une variable ou si une ligne s'exécute, affichez-la avec `console.log()`.

Les erreurs apparaissent en rouge, avec le **nom du fichier** et le **numéro de ligne**. Lisez-les : elles disent presque toujours où chercher.

8.3 Les variables

Une **variable** est une boîte avec une étiquette, dans laquelle on range une valeur pour la réutiliser plus tard.

```
const prenom = "David";
let age = 21;

console.log(prenom); // affiche : David
age = 22;            // on change la valeur
console.log(age);    // affiche : 22
```

Il existe deux mots-clés pour créer une variable :

- `const` crée une variable qui ne sera **pas réassignée**. Utilisez-le par défaut ;
- `let` crée une variable qui **pourra changer**, comme un compteur.

Vous verrez aussi `var` dans d'anciens tutoriels. C'est l'ancienne manière de faire ; elle a des comportements surprenants. Utilisez `const` et `let`.

Les deux barres obliques `//` introduisent un **commentaire** d'une ligne, ignoré par le navigateur.

Les types de valeurs

TYPE	EXEMPLE	DESCRIPTION
Chaîne de caractères (*string*)	`"Bonjour"`, `'Kisangani'`	Du texte, entre guillemets
Nombre (*number*)	`42`, `3.14`	Entiers et décimaux, sans guillemets
Booléen (*boolean*)	`true`, `false`	Vrai ou faux
Tableau (*array*)	`["HTML", "CSS"]`	Une liste de valeurs
Objet (*object*)	`{ nom: "David", age: 21 }`	Des valeurs nommées

Attention : `"42"` (avec guillemets) est un texte, `42` (sans guillemets) est un nombre. `"2" + "2"` donne `"22"`, alors que `2 + 2` donne `4`. Cette différence est à l'origine de nombreux bugs.

Assembler du texte

```
const prenom = "Amina";
const message = `Bonjour ${prenom}, bienvenue !`;
console.log(message); // Bonjour Amina, bienvenue !
```

Les **accents graves** `` ` `` (touche `Alt Gr + 7` sur un clavier français PC) permettent d'insérer une variable dans un texte avec `${ }`. C'est ce qu'on appelle un *template literal*.

8.4 Les conditions

Une **condition** permet au programme de prendre une décision : « si ceci est vrai, fais cela, sinon fais autre chose ».

```
const age = 18;

if (age >= 18) {
  console.log("Majeur");
} else {
  console.log("Mineur");
}
```

OPERATEUR	SIGNIFICATION	EXEMPLE VRAI
`===`	Strictement égal	`5 === 5`
`!==`	Différent	`5 !== 3`
`>` / `<`	Supérieur / inférieur	`10 > 3`
`>=` / `<=`	Supérieur ou égal / inférieur ou égal	`18 >= 18`
`&&`	ET : les deux conditions sont vraies	`age > 12 && age < 20`
`||`	OU : au moins une condition est vraie	`jour === "samedi" || jour === "dimanche"`
`!`	NON : inverse la condition	`!estConnecte`

! = N'EST PAS ===

Un seul `=` **range** une valeur dans une variable. Trois `===` **comparent** deux valeurs. Écrire `if (age = 18)` est une erreur classique : cela modifie `age` au lieu de le tester. Utilisez toujours `===` pour comparer.

8.5 Les fonctions

Une **fonction** est une suite d'instructions à laquelle on donne un nom, pour pouvoir l'exécuter quand on veut, autant de fois qu'on veut.

```
function saluer(nom) {
  return `Bonjour ${nom} !`;
}

console.log(saluer("Joël"));   // Bonjour Joël !
console.log(saluer("Grâce"));  // Bonjour Grâce !
```

- `nom` est un **paramètre** : une valeur que l'on donne à la fonction quand on l'appelle ;
- `return` renvoie un résultat ;
- `saluer("Joël")` **appelle** la fonction.

Vous verrez aussi une écriture plus courte, appelée **fonction fléchée** :

```
const saluer = (nom) => {
  return `Bonjour ${nom} !`;
};
```

Les deux écritures font la même chose dans ce cas. Les fonctions fléchées sont très utilisées pour réagir aux événements, comme vous le verrez au chapitre suivant.

8.6 Tableaux et boucles

Un **tableau** range plusieurs valeurs dans une seule variable. On accède à chaque valeur par sa position, en commençant à **zéro**.

```
const competences = ["HTML", "CSS", "JavaScript"];

console.log(competences[0]);      // HTML
console.log(competences.length);  // 3

competences.push("Git");          // ajoute à la fin
```

Une **boucle** répète une action pour chaque élément :

```
for (const competence of competences) {
  console.log(`J'apprends ${competence}`);
}
```

Traduction : « pour chaque compétence du tableau `competences`, affiche une phrase ». La boucle `for…of` est la plus lisible pour parcourir un tableau.

> **À RETENIR**
>
> Les variables **rangent**, les conditions **décident**, les fonctions **regroupent**, les boucles **répètent**. Avec ces quatre outils, vous pouvez déjà écrire une grande partie des programmes que vous rencontrerez.

EXERCICES

1. Reliez `script.js` à votre portfolio et affichez un message dans la console.
2. Créez une variable contenant votre prénom et une autre votre ville. Affichez une phrase qui utilise les deux.
3. Écrivez une condition qui affiche « Bonjour » si l'heure est inférieure à 18, et « Bonsoir » sinon. Indice : `new Date().getHours()` donne l'heure actuelle.
4. Écrivez une fonction `double(nombre)` qui renvoie le double d'un nombre. Testez-la avec trois valeurs.
5. Créez un tableau de vos cinq aliments préférés et affichez-les un par un avec une boucle.
6. Provoquez volontairement une erreur (une faute de frappe dans un nom de variable). Lisez le message dans la console : quel fichier, quelle ligne ?

CHAPITRE 09

Le DOM et les événements

JavaScript devient vraiment utile quand il agit sur la page elle-même.

9.1 Qu'est-ce que le DOM ?

Quand le navigateur lit votre HTML, il construit en mémoire une représentation de la page sous forme d'**arbre** : le document contient `html`, qui contient `head` et `body`, qui contient `header`, qui contient `h1`… Cet arbre s'appelle le **DOM**, pour *Document Object Model*.

JavaScript peut lire et modifier cet arbre. Changer le DOM, c'est changer la page affichée, instantanément, sans la recharger. L'onglet *Éléments* des outils de développement vous montre justement le DOM, et non le fichier HTML d'origine.

9.2 Sélectionner un élément

Avant de modifier un élément, il faut le trouver. La méthode `document.querySelector()` utilise **les mêmes sélecteurs que CSS** :

```
const titre = document.querySelector("h1");
const bouton = document.querySelector("#bonjour");
const premiereCarte = document.querySelector(".carte");
```

`querySelector` renvoie le **premier** élément qui correspond. Pour obtenir **tous** les éléments, utilisez `querySelectorAll` :

```
const cartes = document.querySelectorAll(".carte");
console.log(cartes.length); // le nombre de cartes
```

Si aucun élément ne correspond, `querySelector` renvoie `null`. Essayer ensuite de modifier `null` provoque l'erreur la plus célèbre des débutants : *Cannot read properties of null*. Quand vous la voyez, vérifiez d'abord votre sélecteur.

9.3 Modifier le contenu et le style

```
const titre = document.querySelector("h1");

titre.textContent = "Mon nouveau titre";  // change le texte
titre.classList.add("mise-en-avant");     // ajoute une classe
titre.classList.remove("mise-en-avant");  // retire une classe
titre.classList.toggle("mise-en-avant");  // ajoute ou retire selon le cas
```

Pour changer l'apparence, la bonne pratique consiste à **ajouter ou retirer une classe CSS**, plutôt qu'à modifier le style directement depuis JavaScript. Le style reste dans le CSS, le comportement dans le JavaScript : chacun son rôle.

Pourquoi préférer textContent ?

Il existe deux façons courantes de changer le contenu d'un élément : `textContent` et `innerHTML`.

- `textContent` insère du **texte pur**. Si le texte contient `<strong>`, il s'affichera tel quel, avec les chevrons ;
- `innerHTML` interprète le texte **comme du HTML**.

Pour afficher un texte fourni par un utilisateur ou par une source externe, utilisez `textContent`. Avec `innerHTML`, une personne malveillante pourrait saisir du code dans un champ de formulaire et le faire exécuter dans la page. Cette faille s'appelle le **XSS** (*cross-site scripting*) ; vous l'étudierez en détail dans le tome 2. Retenez dès maintenant le bon réflexe : **une donnée reste une donnée, elle ne doit pas devenir du code**.

9.4 Écouter les événements

Un **événement** est quelque chose qui se produit dans la page : un clic, une touche pressée, un formulaire envoyé, la souris qui survole un élément. JavaScript peut **écouter** un événement et exécuter une fonction quand il se produit.

```
<button id="bonjour">Dire bonjour</button>
<p id="message"></p>
```

```
const bouton = document.querySelector("#bonjour");
const message = document.querySelector("#message");

bouton.addEventListener("click", () => {
  message.textContent = "Bonjour et bienvenue sur mon portfolio !";
});
```

Lisons ce code comme une phrase : « Sur le bouton, ajoute un écouteur d'événement : quand il y a un **clic**, exécute cette fonction, qui change le texte du message. »

ÉVENEMENT	QUAND IL SE PRODUIT
`Click`	Un clic (ou un toucher sur mobile)
`Submit`	L'envoi d'un formulaire
`Input`	Une modification dans un champ, à chaque caractère
`Keydown`	Une touche du clavier est enfoncée
`Mouseenter`	La souris entre sur l'élément

Un compteur

```
<button id="plus">+1</button>
<p>Clics : <span id="compteur">0</span></p>
```

```
const boutonPlus = document.querySelector("#plus");
const affichage = document.querySelector("#compteur");
let total = 0;

boutonPlus.addEventListener("click", () => {
  total = total + 1;
  affichage.textContent = total;
});
```

Remarquez que `total` est déclaré avec `let` : sa valeur change à chaque clic.

9.5 Réagir à un formulaire

```
<form id="formulaire-contact">
  <label for="nom">Nom</label>
  <input id="nom" name="nom" type="text">
  <button type="submit">Envoyer</button>
</form>
<p id="resultat"></p>
```

```
const form = document.querySelector("#formulaire-contact");
const resultat = document.querySelector("#resultat");

form.addEventListener("submit", (event) => {
  event.preventDefault();

  const nom = document.querySelector("#nom").value.trim();

  if (nom === "") {
    resultat.textContent = "Merci d'indiquer votre nom.";
    return;
  }

  resultat.textContent = `Merci ${nom}, votre message a bien été préparé.`;
});
```

Trois nouveautés :

- `event.preventDefault()` empêche le comportement par défaut du formulaire, qui serait de recharger la page. Sans cette ligne, votre message s'afficherait une fraction de seconde puis disparaîtrait ;
- `.value` lit ce que l'utilisateur a tapé dans le champ ;
- `.trim()` supprime les espaces au début et à la fin. Ainsi, un nom composé uniquement d'espaces est considéré comme vide.

Le `return` arrête la fonction : si le nom est vide, on affiche l'erreur et on ne va pas plus loin.

> **! LA VALIDATION DANS LE NAVIGATEUR NE SUFFIT PAS**
>
> Vérifier un formulaire en JavaScript améliore le confort de l'utilisateur, mais ne protège rien : n'importe qui peut désactiver JavaScript ou envoyer des données directement à un serveur. Le jour où votre formulaire enverra vraiment des données, le **serveur** devra les vérifier à nouveau. C'est l'un des principes de sécurité du tome 2.

EXERCICES

1. Ajoutez un bouton « Afficher mon nom » qui écrit votre nom dans un paragraphe.
2. Créez un compteur qui augmente à chaque clic, puis ajoutez un bouton « Remettre à zéro ».
3. Faites changer une classe CSS lorsqu'un bouton est cliqué (par exemple, mettre une carte en surbrillance).
4. Ajoutez une validation simple à votre formulaire de contact : le nom et le message ne doivent pas être vides.
5. Testez ce qui se passe lorsque le champ ne contient que des espaces, avec et sans `.trim()`.
6. Testez `innerHTML` et `textContent` en affichant le texte `<strong>test</strong>`. Expliquez la différence observée.

CHAPITRE 10

Mini-projet JavaScript : un portfolio interactif

Transformez votre portfolio en petite application, avec trois interactions réelles.

Vous allez ajouter à votre portfolio :

1. un **message de bienvenue** adapté à l'heure de la journée ;
2. un **bouton de thème clair / sombre**, qui se souvient du choix du visiteur ;
3. une **validation complète** du formulaire de contact.

10.1 Le message de bienvenue

Dans le HTML, ajoutez dans le `header` :

```
<p id="bienvenue"></p>
```

Dans `script.js` :

```
function messageSelonHeure() {
  const heure = new Date().getHours();

  if (heure < 12) {
    return "Bonjour, belle matinée pour apprendre !";
  } else if (heure < 18) {
    return "Bon après-midi, merci de votre visite.";
  } else {
    return "Bonsoir, merci de passer par ici.";
  }
}

document.querySelector("#bienvenue").textContent = messageSelonHeure();
```

`else if` permet d'enchaîner plusieurs conditions. Le navigateur les teste dans l'ordre et s'arrête à la première qui est vraie.

10.2 Le thème clair / sombre

C'est ici que les variables CSS du chapitre 7 montrent toute leur puissance. Dans `style.css`, ajoutez :

```
body.sombre {
  --couleur-fond: #111110;
  --couleur-texte: #f5f5f2;
  --couleur-ligne: #333330;
}

body.sombre .carte {
  background-color: #1c1c1a;
}
```

Quand le `body` reçoit la classe `sombre`, les variables changent de valeur, et **toute la page** change de couleurs. Il ne reste qu'à ajouter ou retirer cette classe avec JavaScript.

Dans le HTML, ajoutez un bouton dans le `header` :

```
<button id="theme" class="bouton" type="button">Mode sombre</button>
```

Dans `script.js` :

```
const boutonTheme = document.querySelector("#theme");

function appliquerTheme(theme) {
  const estSombre = theme === "sombre";
  document.body.classList.toggle("sombre", estSombre);
  boutonTheme.textContent = estSombre ? "Mode clair" : "Mode sombre";
}

boutonTheme.addEventListener("click", () => {
  const nouveauTheme = document.body.classList.contains("sombre") ? "clair" :
"sombre";
  appliquerTheme(nouveauTheme);
  localStorage.setItem("theme", nouveauTheme);
});

appliquerTheme(localStorage.getItem("theme") || "clair");
```

Quelques explications :

- `classList.toggle("sombre", estSombre)` ajoute la classe si `estSombre` vaut `true`, et la retire s'il vaut `false` ;
- `condition ? valeurSiVrai : valeurSiFaux` est une **condition courte**, appelée opérateur ternaire. Elle remplace un `if…else` simple ;

- `localStorage` est une petite mémoire du navigateur. `setItem` y range une valeur, `getItem` la relit. Le choix du visiteur est ainsi conservé quand il revient sur la page ;
- `localStorage.getItem("theme") || "clair"` signifie : « le thème enregistré, ou à défaut, le thème clair ».

10.3 Une validation complète du formulaire

```
const form = document.querySelector("#formulaire-contact");
const retour = document.querySelector("#retour-formulaire");

form.addEventListener("submit", (event) => {
  event.preventDefault();

  const nom = form.nom.value.trim();
  const email = form.email.value.trim();
  const message = form.message.value.trim();
  const erreurs = [];

  if (nom === "") erreurs.push("Le nom est obligatoire.");
  if (!email.includes("@")) erreurs.push("L'adresse e-mail semble incorrecte.");
  if (message.length < 10) erreurs.push("Le message doit contenir au moins 10 carac-
tères.");

  if (erreurs.length > 0) {
    retour.textContent = erreurs.join(" ");
    retour.className = "retour erreur";
    return;
  }

  retour.textContent = `Merci ${nom} ! Votre message est prêt à être envoyé.`;
  retour.className = "retour succes";
  form.reset();
});
```

Ajoutez `<p id="retour-formulaire" class="retour" aria-live="polite"></p>` sous le formulaire, et deux classes CSS :

```
.retour.erreur { color: #a11b1b; }
.retour.succes { color: #2f6b3b; }
```

L'attribut `aria-live="polite"` demande aux lecteurs d'écran d'annoncer le message quand il change. Une petite ligne qui rend votre formulaire utilisable par tous.

`form.nom` est un raccourci : il désigne le champ du formulaire dont l'attribut `name` vaut `nom`. `form.reset()` vide tous les champs après un envoi réussi.

> **ENVOYER REELLEMENT LE FORMULAIRE**
>
> Pour l'instant, le message n'est envoyé nulle part. Pour recevoir les messages par e-mail sans écrire de serveur, des services comme Formspree ou Netlify Forms proposent des offres gratuites : vous remplacez l'adresse de l'attribut `action` du formulaire par celle qu'ils vous fournissent. Lisez toujours leurs conditions d'utilisation, notamment sur la protection des données de vos visiteurs.

10.4 Relire son propre code

Avant de passer à la suite, faites un exercice que les professionnels pratiquent chaque jour : la **relecture**. Pour chaque ligne de `script.js`, posez-vous trois questions :

1. Que fait cette ligne ?
2. Que se passe-t-il si l'élément qu'elle cherche n'existe pas ?
3. Pourrais-je la réécrire sans regarder ce livre ?

Si la réponse à la troisième question est non, reprenez l'exemple, effacez-le et réécrivez-le de mémoire, quitte à vous tromper. C'est la différence entre **reconnaître** un code et le **posséder**.

> **AVEC L'IA**
>
> Demandez à un assistant IA : « Pose-moi cinq questions pour vérifier que je comprends ce code JavaScript », puis collez votre script. Répondez sans regarder vos notes. L'IA devient alors un examinateur, et non un écrivain fantôme. C'est l'un des meilleurs usages possibles de ces outils quand on apprend.

EXERCICES

1. Ajoutez le message de bienvenue selon l'heure.
2. Créez le bouton de thème clair / sombre et vérifiez que le choix est conservé après un rechargement.
3. Empêchez l'envoi du formulaire si le nom est vide ou si l'e-mail ne contient pas de `@`.
4. Affichez un message de réussite après une validation correcte et videz les champs.
5. Ajoutez une quatrième interaction de votre choix : un bouton « Retour en haut », une citation aléatoire, un compteur de projets…
6. Expliquez chaque ligne de votre JavaScript à voix haute, sans la lire dans ce livre.

IV

PARTIE IV

Git

Suivre l'histoire de votre code : enregistrer chaque étape, comparer, revenir en arrière.

CHAPITRE 11

Qu'est-ce que Git ?

Git est votre filet de sécurité. Avec lui, vous pouvez tout essayer, parce que vous pouvez toujours revenir en arrière.

Imaginez que votre portfolio fonctionne parfaitement. Vous décidez de refaire la navigation. Une heure plus tard, plus rien ne marche, et vous ne vous souvenez plus de ce que vous avez modifié. Sans outil, vous êtes perdu. Avec Git, il suffit de demander : « Montre-moi ce qui a changé depuis la dernière version qui fonctionnait », puis éventuellement : « Reviens à cette version. »

11.1 Le principe

Git est un **système de gestion de versions**. Pensez à une machine à remonter le temps pour votre projet, ou à un album photo de votre chantier : à chaque étape importante, vous prenez une photo de l'ensemble de vos fichiers. Vous pouvez ensuite feuilleter l'album, comparer deux photos et revenir à n'importe laquelle.

Git a été créé en 2005 par Linus Torvalds, le créateur de Linux. Il est aujourd'hui utilisé par la quasi-totalité des équipes de développement dans le monde, de la petite agence aux plus grandes entreprises.

Git et GitHub ne sont pas la même chose

C'est la confusion la plus fréquente :

- **Git** est un **logiciel** installé sur votre ordinateur. Il fonctionne **localement**, même sans Internet ;
- **GitHub** est une **plateforme en ligne** qui héberge des projets Git, permet de les partager, de collaborer et de les publier.

Git est l'appareil photo ; GitHub est l'album partagé en ligne. Vous pouvez utiliser Git sans GitHub, mais pas GitHub sans Git. Nous verrons GitHub dans la partie suivante.

11.2 Les mots à connaître

MOT	SIGNIFICATION
Dépôt (*repository*, « repo »)	Un dossier de projet suivi par Git

MOT	SIGNIFICATION
Commit	Une « photo » enregistrée dans l'historique, avec un message qui la décrit
Branche (*branch*)	Une ligne de travail séparée, pour essayer sans risque
Cloner (*clone*)	Copier un dépôt distant sur votre machine
Pousser (*push*)	Envoyer vos commits vers un dépôt distant, par exemple sur GitHub
Tirer (*pull*)	Récupérer les commits présents sur le dépôt distant

11.3 Installer Git

C'est maintenant, au moment où vous comprenez son rôle, que vous installez Git.

Sur Windows

1. Rendez-vous sur git-scm.com/downloads et téléchargez la version Windows.
2. Lancez l'installateur. Les écrans sont nombreux : **les options par défaut conviennent** dans presque tous les cas.
3. Sur l'écran du choix de l'éditeur par défaut, vous pouvez sélectionner **Visual Studio Code**.
4. Terminez l'installation. Git installe aussi **Git Bash**, un terminal que vous pouvez utiliser pour taper les commandes.

Sur Mac

1. Ouvrez l'application **Terminal** (dans Applications → Utilitaires).
2. Tapez `git --version` et appuyez sur Entrée.
3. Si Git n'est pas installé, macOS vous propose d'installer les **outils de ligne de commande**. Acceptez et patientez.

Les instructions officielles détaillées pour chaque système sont sur git-scm.com/downloads.

> **VOUS PREFEREZ UNE INTERFACE GRAPHIQUE ?**
>
> GitHub Desktoppermet de faire les opérations Git courantes avec des boutons, sans taper de commandes. C'est une bonne alternative si le terminal vous intimide. Nous vous conseillons cependant d'apprendre d'abord les commandes : elles fonctionnent partout et vous aident à comprendre ce que fait réellement Git. VS Code intègre aussi un panneau Git très pratique (l'icône en forme de branche, dans la barre de gauche).

11.4 Le terminal en cinq minutes

Git s'utilise principalement dans un **terminal** : une fenêtre où l'on tape des commandes au lieu de cliquer. Cela paraît austère au début, mais c'est rapide et précis.

Dans VS Code, ouvrez le terminal intégré avec le menu **Terminal** → **Nouveau terminal** (ou `Ctrl+`). Le terminal s'ouvre directement **dans le dossier de votre projet**, ce qui évite bien des erreurs.

Trois commandes suffisent pour commencer :

COMMANDE	ROLE
`Pwd`	Affiche le dossier où vous êtes (*print working directory*)
`Ls`	Liste les fichiers du dossier (sous l'invite classique de Windows, `dir`)
`cd nom-du-dossier`	Entre dans un dossier. `cd ..` remonte d'un niveau

11.5 La première configuration

Avant votre premier commit, présentez-vous à Git. Ces informations seront inscrites dans chaque commit, pour savoir qui a fait quoi.

```
git --version
git config --global user.name "Votre Nom"
git config --global user.email "vous@example.com"
git config --global init.defaultBranch main
```

- la première commande vérifie que Git est installé et affiche sa version ;
- les deux suivantes enregistrent votre nom et votre e-mail. Utilisez la **même adresse e-mail** que celle de votre futur compte GitHub ;
- la dernière nomme `main` la branche principale de vos nouveaux projets, comme le fait GitHub.

L'option `--global` signifie que ces réglages s'appliquent à tous vos projets. Vous ne les tapez qu'une fois par ordinateur.

GitHub propose un guide officiel en français pour cette étape : configurer Git.

EXERCICES

1. Installez Git sur votre ordinateur.
2. Vérifiez l'installation avec `git --version`.
3. Configurez votre nom, votre e-mail et la branche par défaut.
4. Ouvrez le terminal de VS Code dans votre dossier de portfolio et testez `pwd` et `ls`.
5. Expliquez à quelqu'un, en deux phrases, la différence entre Git et GitHub.

CHAPITRE 12

Les commandes Git essentielles

Une dizaine de commandes couvrent l'essentiel du travail quotidien d'un développeur.

12.1 Les trois zones de Git

Pour comprendre Git, il faut visualiser trois zones :

1. **Le dossier de travail** : vos fichiers, tels que vous les modifiez dans VS Code.
2. **La zone de préparation** (*staging area*) : les modifications que vous avez choisi d'inclure dans le prochain commit. C'est comme poser des objets sur la table avant la photo.
3. **L'historique** : les commits enregistrés, les photos de l'album.

Le cycle est toujours le même : **modifier → préparer (`add`) → enregistrer (`commit`)**.

COMMANDE	À QUOI ELLE SERT
`git init`	Commencer le suivi Git dans un dossier
`git status`	Voir l'état du projet : ce qui a changé, ce qui est préparé
`git add fichier`	Préparer un fichier pour le prochain commit
`git add .`	Préparer **toutes** les modifications du dossier
`git commit -m "message"`	Enregistrer une photo dans l'historique
`git log --oneline`	Consulter l'historique, une ligne par commit
`git diff`	Voir précisément ce qui a changé et n'est pas encore préparé
`git restore fichier`	Annuler les modifications non enregistrées d'un fichier
`git branch`	Lister les branches
`git switch -c nom`	Créer une branche et s'y placer
`git switch main`	Revenir sur la branche principale

12.2 Votre premier cycle

Dans le terminal, placé dans le dossier de votre portfolio :

```
git init
git status
git add .
git commit -m "Création de mon portfolio"
```

- `git init` crée un dossier caché `.git` : c'est là que Git range tout l'historique. **Ne le supprimez jamais**, ou vous perdrez l'historique ;
- `git status` affiche vos fichiers en rouge : ils sont *non suivis* (*untracked*) ;
- après `git add .`, `git status` les affiche en vert : ils sont *préparés* (*staged*) ;
- `git commit` les enregistre. L'historique contient maintenant un commit.

Modifiez ensuite quelque chose, par exemple ajoutez une section, puis :

```
git status
git diff
git add .
git commit -m "Ajout de la section projets"
git log --oneline
```

`git log --oneline` affiche vos deux commits, du plus récent au plus ancien, chacun précédé d'un code unique (par exemple `a3f9c21`). Ce code est l'identifiant du commit.

> ■ **À RETENIR**
>
> Faites de `git status` un réflexe. Tapez-le avant et après chaque commande : il vous dit toujours où vous en êtes et, souvent, quelle commande taper ensuite.

12.3 Écrire de bons messages de commit

Un commit raconte une **étape cohérente** de l'évolution du projet. Son message doit permettre à quelqu'un, vous y compris dans six mois, de comprendre ce qui a changé sans ouvrir le code.

MESSAGE FAIBLE	MESSAGE CLAIR
`modif`	`Ajout du formulaire de contact`
`fix`	`Correction du menu qui débordait sur mobile`
`update`	`Passage des couleurs en variables CSS`
`aaaa`	`Ajout du bouton de thème clair / sombre`

Trois règles simples :

- commencez par un **verbe ou un nom d'action** : *Ajout*, *Correction*, *Suppression*, *Réorganisation* ;

- un commit = **une idée**. Si votre message contient « et » trois fois, faites plusieurs commits ;
- **commitez souvent** : à chaque étape qui fonctionne. Un petit commit se comprend et s'annule facilement.

12.4 Ignorer certains fichiers

Certains fichiers ne doivent pas être suivis par Git : des fichiers système, des brouillons, plus tard des mots de passe ou des clés secrètes. Créez à la racine du projet un fichier nommé exactement `.gitignore` :

```
# Fichiers système
.DS_Store
Thumbs.db

# Brouillons personnels
brouillons/
```

Chaque ligne indique un fichier ou un dossier que Git doit ignorer. Les lignes qui commencent par `#` sont des commentaires.

12.5 Revenir en arrière, simplement

Git permet beaucoup de retours en arrière. Voici les deux plus utiles pour débuter, et les plus sûrs :

Annuler des modifications pas encore commitées sur un fichier :

```
git restore index.html
```

Le fichier revient à son état du dernier commit. Attention : les modifications annulées sont perdues.

Consulter une ancienne version sans rien casser : `git log --oneline` vous donne l'identifiant des commits, et `git show a3f9c21` affiche ce qui a changé dans ce commit.

Il existe des commandes plus puissantes pour réécrire l'historique. Ne les utilisez pas tant que vous ne les comprenez pas parfaitement : elles peuvent faire perdre du travail. Le tome 2 y revient en détail.

12.6 Les branches

Une **branche** est une ligne de travail parallèle. Elle vous permet d'essayer une idée sans toucher à la version principale.

```
git switch -c design
# … modifiez votre CSS …
git add .
git commit -m "Essai d'une nouvelle palette de couleurs"
git switch main
```

Sur la branche `design`, votre nouvelle palette existe. Revenez sur `main` : votre portfolio retrouve ses anciennes couleurs, comme si rien ne s'était passé. Vos fichiers changent **réellement** sous vos yeux dans VS Code. Si l'essai vous plaît, vous pouvez l'intégrer à `main` :

```
git switch main
git merge design
```

`git merge` fusionne le travail de la branche `design` dans la branche où vous vous trouvez.

> **! COMMITEZ AVANT DE CHANGER DE BRANCHE**
>
> Si vous avez des modifications non enregistrées, Git peut refuser de changer de branche, ou les emporter avec vous. Prenez l'habitude : `git status`, puis commit, puis changement de branche.

EXERCICES

1. Initialisez Git dans le dossier de votre portfolio et faites un premier commit.
2. Faites trois modifications différentes et un commit après chacune, avec un message clair.
3. Utilisez `git log --oneline` pour voir vos commits.
4. Modifiez un fichier, observez le changement avec `git diff`, puis annulez-le avec `git restore`.
5. Créez un fichier `.gitignore`.
6. Créez une branche nommée `design`, faites-y une modification et un commit.
7. Revenez sur `main`, constatez la différence, puis fusionnez la branche avec `git merge design`.

V

PARTIE V

GitHub

Publier et partager votre travail : sauvegarder en ligne, présenter vos projets, mettre votre site en ligne.

CHAPITRE 13

Comprendre GitHub

GitHub est à la fois votre coffre-fort, votre vitrine et votre hébergeur gratuit.

GitHub est une plateforme en ligne qui héberge des dépôts Git. Elle vous permet de :

- **sauvegarder** votre code en ligne : si votre ordinateur tombe en panne, votre travail est en sécurité ;
- **présenter** vos projets : votre profil GitHub est un CV technique que les recruteurs et les clients consultent ;
- **collaborer** avec d'autres développeurs ;
- **publier** gratuitement un site statique, comme votre portfolio, grâce à **GitHub Pages**.

Pour suivre ce chapitre, Git doit être installé et configuré (chapitre 11), et votre portfolio doit avoir au moins un commit (chapitre 12).

13.1 Créer votre compte

1. Rendez-vous sur github.com et cliquez sur **Sign up**.
2. Utilisez la **même adresse e-mail** que celle configurée dans Git.
3. Choisissez votre **nom d'utilisateur** avec soin : il apparaîtra dans l'adresse de vos projets et de votre site (`votre-nom.github.io`). Préférez un nom simple et professionnel, proche de votre vrai nom.
4. Activez l'**authentification à deux facteurs** lorsque GitHub vous le propose. Elle protège votre compte même si votre mot de passe est volé.

GitHub propose un parcours de découverte en français : démarrer avec GitHub.

13.2 Créer votre premier dépôt

1. Une fois connecté, cliquez sur le **+** en haut à droite, puis **New repository**.
2. Donnez un nom clair : `portfolio`.
3. Ajoutez une courte description : « Mon premier portfolio, construit en HTML, CSS et JavaScript. »
4. Choisissez **Public** : c'est nécessaire pour publier gratuitement avec GitHub Pages, et c'est ce qui permet de montrer votre travail.

5. **Ne cochez pas** *Add a README file* : votre projet existe déjà sur votre ordinateur, vous allez l'envoyer.
6. Cliquez sur **Create repository**.

GitHub affiche alors une page d'instructions. Gardez-la ouverte.

13.3 Relier votre projet local à GitHub

Dans le terminal de VS Code, dans le dossier de votre portfolio :

```
git remote add origin https://github.com/VOTRE-NOM/portfolio.git
git branch -M main
git push -u origin main
```

- `git remote add origin …` indique à Git l'adresse de votre dépôt en ligne et lui donne le surnom `origin` ;
- `git branch -M main` s'assure que votre branche principale s'appelle `main` ;
- `git push -u origin main` envoie vos commits. L'option `-u` mémorise la destination : les fois suivantes, un simple `git push` suffira.

Rechargez la page de votre dépôt sur GitHub : vos fichiers y sont.

La question de l'authentification

Au premier `push`, GitHub doit vérifier que c'est bien vous. **Votre mot de passe GitHub ne fonctionne pas dans le terminal** : GitHub l'a interdit pour des raisons de sécurité.

- **Sur Windows**, Git est livré avec un gestionnaire d'identifiants qui ouvre une fenêtre de connexion. Choisissez la connexion par le navigateur, validez, et c'est terminé : l'autorisation est mémorisée.
- **Sur Mac**, le plus simple est d'utiliser GitHub Desktop pour le premier envoi, ou l'outil en ligne de commande officiel de GitHub. Vous pouvez aussi créer un **jeton d'accès personnel** dans les paramètres de votre compte, qui remplace le mot de passe.

Le guide configurer Git détaille toutes les options, y compris les clés SSH que vous utiliserez probablement plus tard.

> **NE PUBLIEZ JAMAIS UN SECRET**
>
> Tout ce qui est poussé dans un dépôt public est visible par le monde entier, **y compris dans l'historique**, même si vous supprimez le fichier ensuite. Ne mettez jamais dans un dépôt un mot de passe, une clé d'API ou des informations personnelles. Pour l'instant votre portfolio n'en contient pas, mais prenez le réflexe dès aujourd'hui.

13.4 Le cycle quotidien

Désormais, votre travail suit toujours le même rythme :

```
git status
git add .
git commit -m "Description claire de la modification"
git push
```

Si vous travaillez sur deux appareils, ou si vous avez modifié un fichier directement sur GitHub, récupérez d'abord les changements avec `git pull` avant de commencer.

13.5 Le README : la vitrine de votre projet

Le fichier `README.md` est la première chose que les visiteurs voient sur la page de votre dépôt. Il est écrit en **Markdown**, une syntaxe très simple pour mettre en forme du texte.

```
# Mon portfolio

Mon premier site personnel, construit à la main pendant
ma formation avec « L'avantage du développeur ».

## Technologies
- HTML sémantique
- CSS (Flexbox, variables, responsive)
- JavaScript (DOM, événements, localStorage)

## Ce que j'ai appris
- Structurer une page avec des éléments sémantiques
- Rendre une page lisible sur téléphone
- Versionner mon travail avec Git

## Voir le site
https://votre-nom.github.io/portfolio/
```

En Markdown, `#` crée un titre, `##` un sous-titre et `-` une puce. Créez ce fichier dans VS Code, puis commit et push.

13.6 Publier votre site avec GitHub Pages

C'est le moment le plus satisfaisant du livre : mettre votre portfolio **en ligne**, à une adresse que vous pouvez partager avec n'importe qui.

1. Sur la page de votre dépôt, cliquez sur **Settings** (Paramètres).
2. Dans le menu de gauche, cliquez sur **Pages**.
3. Dans la partie *Build and deployment*, choisissez la source **Deploy from a branch**.
4. Sélectionnez la branche **main** et le dossier **/ (root)**, puis cliquez sur **Save**.
5. Patientez une à deux minutes, puis rechargez la page : GitHub affiche l'adresse de votre site, de la forme `https://votre-nom.github.io/portfolio/`.

Votre fichier doit s'appeler exactement `index.html` et se trouver à la racine du dépôt. Ensuite, chaque `git push` met automatiquement votre site à jour, en général en moins d'une minute.

Guide officiel : créer un site GitHub Pages.

> **UN SITE A L'ADRESSE DE VOTRE NOM**
>
> Si vous nommez un dépôt exactement `votre-nom.github.io` (avec votre nom d'utilisateur), GitHub Pages publie son contenu à la racine : `https://votre-nom.github.io/`, sans `/portfolio/` à la fin. C'est l'adresse idéale pour un portfolio.

13.7 GitHub depuis un téléphone

Sur smartphone, vous pouvez consulter vos dépôts dans le navigateur ou avec l'application mobile GitHub, et modifier certains fichiers directement en ligne.

L'éditeur **github.dev** est gratuit et fonctionne dans le navigateur : sur la page de n'importe lequel de vos dépôts, appuyez sur la touche `.` (point) d'un clavier, ou remplacez `github.com` par `github.dev` dans l'adresse. Il ressemble à VS Code et permet de modifier des fichiers et de faire des commits. En revanche, il ne fournit **pas de terminal** et ne peut pas exécuter votre projet.

Pour un environnement complet dans le navigateur, avec un terminal, GitHub Codespaces fournit une machine de développement en ligne. Il inclut un quota d'utilisation gratuit chaque mois pour les comptes personnels : surveillez votre consommation dans les paramètres de facturation.

EXERCICES

1. Créez votre compte GitHub et activez l'authentification à deux facteurs.
2. Créez un dépôt public `portfolio`.
3. Reliez votre projet local et envoyez-le avec `git push`.
4. Rédigez un `README.md` qui présente votre projet, puis envoyez-le.
5. Publiez votre portfolio avec GitHub Pages et ouvrez l'adresse sur votre téléphone.
6. Ouvrez votre dépôt avec github.dev, faites une petite modification et un commit depuis le navigateur. Puis, sur votre ordinateur, récupérez-la avec `git pull`.

CHAPITRE 14

Branches et pull requests, simplement

La pull request est la manière dont les équipes du monde entier proposent, relisent et intègrent du code.

14.1 Pourquoi passer par une pull request ?

Au chapitre 12, vous avez fusionné une branche avec `git merge`. En équipe, on ne fusionne presque jamais directement. On ouvre une **pull request** (souvent abrégée « PR ») : une **proposition de modification**, que d'autres peuvent lire, commenter et approuver avant qu'elle rejoigne la branche principale.

Même si vous travaillez seul, cette méthode est précieuse : elle vous oblige à relire vos changements avant de les intégrer, et elle vous prépare au travail en entreprise.

14.2 Le parcours complet

1. Créez une branche et travaillez dessus :

```
git switch -c section-temoignages
# … ajoutez une section dans index.html …
git add .
git commit -m "Ajout de la section témoignages"
```

2. Envoyez la branche sur GitHub :

```
git push -u origin section-temoignages
```

3. Ouvrez la pull request. Sur la page de votre dépôt, GitHub affiche un bandeau jaune proposant **Compare & pull request**. Cliquez dessus. Donnez un titre clair et décrivez **ce que vous avez changé et pourquoi**.

4. Relisez. L'onglet **Files changed** montre chaque ligne ajoutée (en vert) et supprimée (en rouge). Relisez comme si le code venait de quelqu'un d'autre. C'est souvent à ce moment que l'on repère une faute de frappe ou un oubli.

5. Fusionnez. Si tout est correct, cliquez sur **Merge pull request**, puis **Confirm merge**. Vous pouvez ensuite supprimer la branche en ligne.

6. Mettez à jour votre ordinateur :

```
git switch main
git pull
```

Votre branche `main` locale contient désormais la nouvelle section, et votre site GitHub Pages est mis à jour.

> **À RETENIR**
>
> **Branche → commits → push → pull request → relecture → fusion → pull.** Ce cycle est le cœur du travail en équipe. Le maîtriser seul, sur un petit projet, vous donne une longueur d'avance.

14.3 Les conflits : pas de panique

Un **conflit** survient quand deux branches ont modifié **la même ligne** du même fichier. Git ne peut pas deviner quelle version garder : il vous demande de choisir. Le fichier concerné contient alors des marques comme celles-ci :

```
<<<<<<< HEAD
<h2>Mes réalisations</h2>
=======
<h2>Mes projets</h2>
>>>>>>> section-projets
```

La partie entre `<<<<<<<` et `=======` est votre version actuelle ; celle entre `=======` et `>>>>>>>` vient de l'autre branche. Pour résoudre le conflit :

1. gardez la bonne version (ou un mélange des deux) ;
2. **supprimez les trois lignes de marques** ;
3. enregistrez, puis `git add .` et `git commit`.

VS Code affiche des boutons au-dessus des conflits (*Accept Current Change*, *Accept Incoming Change*…) qui simplifient l'opération. Un conflit n'est pas une erreur : c'est Git qui vous demande votre avis.

> **POUR PRATIQUER EN TOUTE SECURITE**
>
> GitHub propose un tutoriel officiel en français, « Hello World », qui fait parcourir tout le cycle de la pull request sur un dépôt d'entraînement : Hello World sur GitHub. Les cours interactifs de GitHub Skills sont aussi excellents pour s'entraîner.

EXERCICES

1. Créez une branche `contact` depuis `main`.
2. Améliorez votre section contact et faites un commit.
3. Poussez la branche sur GitHub.
4. Ouvrez une pull request avec un titre et une description claire.
5. Relisez l'onglet *Files changed* et notez au moins une chose que vous auriez pu mieux faire.
6. Fusionnez la pull request, puis mettez à jour votre `main` local avec `git pull`.
7. Bonus : provoquez volontairement un conflit en modifiant le même titre sur deux branches, puis résolvez-le.

VI

PARTIE VI

Apprendre avec l'IA

Faire de l'intelligence artificielle un professeur, pas un écrivain fantôme.

CHAPITRE 15

Apprendre avec l'IA sans perdre votre avantage

L'IA peut vous faire progresser deux fois plus vite, ou vous empêcher de progresser du tout. Tout dépend de la manière dont vous l'utilisez.

Vous savez maintenant construire, styliser, animer, versionner et publier une page Web. Vous avez fait tout cela **en comprenant ce que vous faisiez**. C'est précisément ce qui vous permet, à partir d'ici, de tirer le meilleur des assistants d'intelligence artificielle.

15.1 Ce que l'IA fait bien, et ce qu'elle fait mal

Un assistant comme ChatGPT, Claude, Gemini ou Copilot est un programme entraîné sur d'immenses quantités de textes et de code. Il est capable de produire une réponse fluide et plausible à presque n'importe quelle question. **Plausible ne veut pas dire correcte.**

L'IA EST TRES UTILE POUR…	L'IA PEUT VOUS TROMPER QUAND…
Expliquer une notion avec d'autres mots	Elle invente une propriété CSS ou une fonction qui n'existe pas
Traduire un message d'erreur en langage simple	Elle utilise une syntaxe ancienne, abandonnée depuis des années
Vous poser des questions pour vérifier votre compréhension	Elle « corrige » un bug en en créant deux autres
Proposer des idées d'exercices ou de projets	Elle répond avec assurance à une question qu'elle ne comprend pas
Relire votre code et suggérer des améliorations	Elle produit un code trop complexe pour votre niveau

Le danger, pour un débutant, n'est pas que l'IA se trompe. C'est de **ne pas pouvoir s'en rendre compte**. Voilà pourquoi ce livre vous a fait apprendre les bases avant de parler d'IA : vous avez désormais les moyens de juger une réponse.

15.2 La règle d'or : écrire d'abord, demander ensuite

> LA REGLE D'OR
>
> **Essayez toujours par vous-même avant de demander.** Écrivez votre version, même imparfaite, même cassée. Ensuite seulement, demandez à l'IA de l'expliquer, de la critiquer ou de la comparer à une autre solution.

Pourquoi cet ordre est-il si important ? Parce que l'apprentissage se produit au moment où vous **cherchez**. Si l'IA vous donne la réponse avant que vous ayez réfléchi, votre cerveau la lit, la trouve logique et l'oublie. Si vous avez cherché d'abord, la réponse vient combler un manque précis : elle s'installe durablement.

Un repère simple : accordez-vous **vingt minutes** de recherche personnelle (relire le chapitre, consulter la documentation, utiliser la console) avant de demander de l'aide. Ces vingt minutes ne sont jamais perdues, même quand vous ne trouvez pas.

15.3 Cinq façons d'utiliser l'IA comme un professeur

1. Demander une explication, pas une solution.

```
Je débute en CSS. Explique-moi la différence entre margin et
padding avec une image de la vie quotidienne. Ne me donne pas
de code, je veux d'abord comprendre l'idée.
```

2. Traduire une erreur.

```
Voici le message d'erreur de ma console JavaScript :
"Uncaught TypeError: Cannot read properties of null
(reading 'addEventListener') at script.js:4"
Explique-moi ce qu'il signifie et quelles sont les causes
possibles. Ne corrige pas mon code : je veux trouver moi-même.
```

3. Se faire interroger.

```
Je viens d'apprendre les événements en JavaScript. Pose-moi
cinq questions, de la plus simple à la plus difficile, pour
vérifier ma compréhension. Attends ma réponse avant de
passer à la suivante.
```

4. Faire relire son code.

```
Voici mon code HTML. Je suis débutant. Indique-moi les trois
problèmes les plus importants (structure, accessibilité,
bonnes pratiques), explique pourquoi ce sont des problèmes,
mais laisse-moi les corriger.
```

5. Comparer deux approches.

```
J'ai écrit cette fonction de validation de formulaire.
Propose une autre façon de l'écrire et compare les deux :
lisibilité, facilité de modification, risques d'erreur.
```

Remarquez ce que ces demandes ont en commun : elles **précisent votre niveau**, elles **fixent une limite** (« ne me donne pas la solution ») et elles vous laissent **le travail final**.

15.4 Vérifier ce que dit l'IA

Prenez l'habitude de vérifier toute affirmation technique importante à une source fiable. Pour le Web, la référence est MDN Web Docs, la documentation de Mozilla, disponible en français.

Trois vérifications simples :

1. **Testez.** Une propriété CSS ou une fonction JavaScript proposée par l'IA fonctionne-t-elle réellement dans votre navigateur ? La console et les outils de développement tranchent en quelques secondes.
2. **Cherchez dans MDN.** Si la propriété n'y figure pas, elle n'existe probablement pas.
3. **Demandez-vous si vous comprenez.** Si vous ne pouvez pas expliquer une ligne, vous ne devez pas la garder. Demandez une explication, ou réécrivez-la plus simplement.

> **LE CODE QUE VOUS NE COMPRENEZ PAS VOUS APPARTIENT QUAND MEME**
>
> Si vous collez dans votre projet un code généré que vous ne comprenez pas, et qu'il contient une erreur ou une faille, c'est **votre** projet qui est cassé, et c'est **vous** qui devrez le réparer. L'IA ne sera pas là pour en assumer les conséquences.

15.5 Protéger vos informations

Tout ce que vous écrivez dans un assistant IA en ligne est envoyé à l'entreprise qui le fournit. Selon le service et vos réglages, ces conversations peuvent être conservées, voire utilisées pour améliorer les modèles.

- Ne collez jamais de **mot de passe**, de **clé d'API** ou de **code secret**.
- Ne partagez pas d'**informations personnelles** sur d'autres personnes : coordonnées de clients, messages privés, données d'un employeur.
- Lisez les paramètres de confidentialité du service que vous utilisez.

Pour vos exercices et votre portfolio, il n'y a aucun risque. Mais prenez les bons réflexes maintenant : ils deviendront essentiels le jour où vous travaillerez sur le projet d'un client.

15.6 Le test du bon usage

À la fin de chaque séance où vous avez utilisé l'IA, posez-vous une seule question :

> « Suis-je capable de refaire ce que j'ai fait aujourd'hui sans l'IA ? »

Si la réponse est oui, l'IA vous a fait progresser. Si la réponse est non, elle a travaillé à votre place. Dans ce cas, refaites l'exercice sans elle, tout de suite, pendant que c'est encore frais.

C'est le cœur de cette série de livres : l'IA est un formidable **accélérateur**, à condition que ce soit **votre pensée** qu'elle accélère. Le tome 3, *Rester indispensable à l'ère de l'IA*, approfondit ces méthodes pour un usage professionnel.

EXERCICES

1. Choisissez une notion de ce livre qui vous a semblé difficile. Demandez à une IA de vous l'expliquer avec une image de la vie quotidienne, sans code.
2. Provoquez une erreur JavaScript dans votre portfolio. Demandez à l'IA d'expliquer le message, **sans corriger le code**, puis corrigez-le vous-même.
3. Demandez à l'IA de vous poser cinq questions sur Git. Répondez sans regarder le livre. Notez vos erreurs.
4. Demandez à l'IA une propriété CSS pour un effet précis. Vérifiez son existence et son fonctionnement dans MDN et dans votre navigateur.
5. Faites relire votre fichier HTML par l'IA. Pour chaque suggestion, décidez si vous l'acceptez ou non, et **expliquez pourquoi** par écrit.

Ressources pour continuer

Ne cherchez pas à tout apprendre en une semaine. Utilisez ces ressources comme une bibliothèque, au moment où une notion vous bloque.

Documentation et cours

RESSOURCE	POUR QUOI FAIRE	LIEN
MDN Web Docs	La référence pour HTML, CSS et JavaScript, en français	developer.mozilla.org/fr
W3Schools	Cours courts et exemples à modifier en direct (en anglais)	w3schools.com
freeCodeCamp	Parcours complets et gratuits, avec certificats	freecodecamp.org
CodePen	Tester HTML, CSS et JavaScript dans le navigateur	codepen.io
Flexbox Froggy	Apprendre Flexbox en jouant, en français	flexboxfroggy.com
Validateur W3C	Vérifier votre HTML	validator.w3.org
Squoosh	Compresser vos images avant de les publier	squoosh.app

Git et GitHub

RESSOURCE	POUR QUOI FAIRE	LIEN
Git	Télécharger Git et consulter sa documentation	git-scm.com
Pro Git	Le livre de référence sur Git, gratuit et en français	git-scm.com/book/fr/v2
GitHub Docs	La documentation officielle de GitHub, en français	docs.github.com/fr
GitHub Skills	Cours interactifs pour pratiquer Git et GitHub	skills.github.com
GitHub Desktop	Utiliser Git avec une interface graphique	desktop.github.com

Éditeurs de code

RESSOURCE	PLATEFORME	LIEN
VS Code	Windows, macOS, Linux	code.visualstudio.com
Acode	Android	Google Play
TrebEdit	Android	Google Play
Textastic	iPhone, iPad	App Store
Replit	Android, iPhone, navigateur	replit.com/mobile

Formations vidéo recommandées

Deux formations complètes en français pour revoir les notions de ce livre sous une autre forme :

- **Formation HTML et CSS** : regarder sur YouTube
- **Formation JavaScript** : regarder sur YouTube

> **COMMENT REGARDER UNE VIDEO DE FORMATION**
>
> Regardez une leçon, **mettez la vidéo en pause**, reproduisez l'exemple vous-même, puis modifiez-le. Regarder une vidéo sans pratiquer donne une impression de compréhension qui disparaît en quelques jours. Une vidéo de vingt minutes doit vous occuper au moins une heure.

Votre parcours de 30 jours

Une heure par jour, un mois, un portfolio en ligne. Si vous avez moins de temps, étalez le parcours : la régularité compte plus que la vitesse.

JOURS	PROGRAMME	CHAPITRES
1 à 3	Installer l'environnement, comprendre fichiers et dossiers, écrire sa première page HTML	1 et 2
4 à 7	Balises, liens, images, listes, formulaires et structure sémantique. Premier jet du portfolio	2, 3 et 4
8 à 12	CSS : sélecteurs, classes, cascade, couleurs, typographie, box model	5 et 6
13 à 16	Flexbox, responsive design, variables CSS : le portfolio devient professionnel	6 et 7
17 à 21	JavaScript : variables, conditions, fonctions, tableaux, DOM et événements	8 et 9
22 à 24	Rendre le portfolio interactif : message, thème, validation du formulaire	10
25 à 27	Git : installation, configuration, commits, `.gitignore`, branches	11 et 12
28 à 29	GitHub : dépôt, push, README, GitHub Pages, pull requests	13 et 14
30	Publier la version finale, la tester sur téléphone et ordinateur, rédiger ce que vous avez appris	15 et projet final

TENEZ UN JOURNAL DE BORD

Chaque jour, écrivez trois lignes dans un fichier `journal.md` de votre projet : ce que vous avez appris, ce qui vous a bloqué, ce que vous ferez demain. Commitez-le avec le reste. Dans un mois, ce journal vous montrera le chemin parcouru, et il impressionnera tous ceux à qui vous montrerez votre dépôt.

SI VOUS PRENEZ DU RETARD

Ne sautez pas d'étape pour « rattraper ». Reprenez là où vous en étiez. Un jour manqué n'est pas un échec ; abandonner le parcours parce qu'on a manqué un jour, si.

Projet final — Votre premier portfolio

À la fin de ce livre, vous devez avoir un projet réel. Il n'a pas besoin d'être parfait. Il doit être personnel, fonctionnel et publié.

Le cahier des charges

Cochez chaque point. Quand tout est coché, votre projet final est terminé.

HTML

- ☐ Une structure propre avec `header`, `nav`, `main`, plusieurs `sections` et `footer`
- ☐ Un seul `h1`, des titres `h2` et `h3` dans un ordre logique
- ☐ Des images avec un texte `alt` pertinent et un poids raisonnable
- ☐ Un formulaire de contact avec des `label` reliés à chaque champ
- ☐ Aucune erreur dans le validateur du W3C

CSS

- ☐ Une identité visuelle définie par des variables CSS
- ☐ Des cartes, des boutons et une navigation stylisés
- ☐ Au moins une mise en page Flexbox
- ☐ Une adaptation mobile testée sur un vrai téléphone
- ☐ Un contour de focus visible pour la navigation au clavier

JavaScript

- ☐ Au moins deux interactions réelles (thème, validation, message…)
- ☐ Utilisation de `textContent` pour afficher du texte
- ☐ Aucune erreur dans la console

Git et GitHub

- ☐ Plusieurs commits avec des messages compréhensibles
- ☐ Au moins une branche fusionnée par une pull request
- ☐ Un dépôt public avec un `README.md` clair
- ☐ Le site publié avec GitHub Pages

Documentation

- ☐ Une courte explication de ce que vous avez appris

☐ La liste de ce que vous voulez apprendre ensuite

LE VRAI OBJECTIF

À la fin, vous ne devez pas seulement avoir une page qui fonctionne. Vous devez pouvoir **expliquer comment elle fonctionne**, **modifier son code** sans crainte et **retrouver votre travail** grâce à Git et GitHub. C'est cela, votre avantage.

Montrez votre travail

Partagez l'adresse de votre portfolio : à votre famille, à vos amis, sur vos réseaux sociaux. Montrer son travail fait peur la première fois, et c'est pourtant ainsi que l'on reçoit des conseils, des encouragements et parfois ses premières opportunités.

Vous pouvez aussi me l'envoyer sur WhatsApp. Je serai heureux de voir ce que vous avez construit.

Conclusion — Commencez aujourd'hui

N'attendez pas de tout comprendre pour pratiquer. Pratiquez pour comprendre.

Vous pouvez commencer maintenant, même avec un téléphone. Écrivez votre première page HTML. Ajoutez un peu de CSS. Faites réagir un bouton avec JavaScript. Puis, lorsque vous aurez accès à un ordinateur, apprenez Git et GitHub, et commencez à construire vos projets dans de meilleures conditions.

Si vous êtes arrivé jusqu'ici en faisant les exercices, mesurez le chemin parcouru. Il y a quelques semaines, une balise `<h1>` était un symbole mystérieux. Aujourd'hui, vous savez structurer une page, la mettre en forme, la rendre interactive, suivre son histoire et la publier aux yeux du monde. Vous savez lire un message d'erreur, chercher dans une documentation et juger une réponse de l'IA.

Vous avez surtout acquis quelque chose qu'aucun outil ne peut vous donner à votre place : **la compréhension de ce que vous construisez**.

Votre avantage ne viendra pas du nombre de lignes de code que vous connaissez par cœur. Il viendra de votre capacité à **comprendre** ce que vous écrivez, à **résoudre** les problèmes et à **continuer d'apprendre**.

Et maintenant ?

Le développement Web est un domaine immense, et c'est une excellente nouvelle : vous ne vous ennuierez jamais. Voici les prochaines étapes naturelles :

- **construire d'autres petits projets** : une page de recettes, un quiz, une calculatrice, une liste de tâches. Chaque projet consolide ce que vous savez ;
- **approfondir JavaScript** : les fonctions, les tableaux et les objets ouvrent la porte à des applications plus riches ;
- **découvrir le côté serveur** : comment un site enregistre des données, gère des utilisateurs et reste sécurisé.

C'est exactement le programme du **tome 2**, *Construire avec plus de méthode* : l'environnement professionnel, le back-end, la modélisation des données, la lisibilité du code, les tests, la sécurité et la mise en production.

Bonne route. Et souvenez-vous : **pensez avant de générer.**

Annexe A — Glossaire

Les mots du développeur Web, expliqués simplement.

MOT	DEFINITION
Attribut	Information supplémentaire placée dans une balise ouvrante : `href`, `src`, `class`...
Balise	Mot entre chevrons qui indique la nature d'un contenu : `<p>`, `<h1>`...
Box model	Modèle de boîte CSS : contenu, padding, bordure, marge
Branche	Ligne de travail parallèle dans Git
Cascade	Règles qui déterminent quel style CSS l'emporte en cas de conflit
Classe	Nom de groupe donné à des éléments HTML pour les styliser ensemble
Commit	Enregistrement d'une étape dans l'historique Git
Console	Zone des outils de développement où s'affichent messages et erreurs JavaScript
Dépôt	Projet suivi par Git, local ou en ligne
DOM	Représentation de la page en mémoire, que JavaScript peut modifier
Élément	Balise ouvrante + contenu + balise fermante
Événement	Action détectée dans la page : clic, saisie, envoi de formulaire...
Flexbox	Système CSS pour aligner et répartir des éléments
Fonction	Suite d'instructions nommée, réutilisable
GitHub Pages	Service gratuit de GitHub pour publier un site statique
Media query	Règle CSS appliquée seulement sous une condition, par exemple la largeur d'écran
Pull request	Proposition de modification, relue avant d'être fusionnée
Push / Pull	Envoyer / récupérer des commits vers / depuis un dépôt distant
Responsive	Qui s'adapte à toutes les tailles d'écran
Sélecteur	Partie d'une règle CSS qui désigne les éléments ciblés
Sémantique	Qui porte un sens : `<nav>` est sémantique, `<div>` ne l'est pas
Variable	Boîte nommée qui contient une valeur

Annexe B — Quand ça ne marche pas

La méthode de débogage à suivre, dans l'ordre, avant de demander de l'aide.

La méthode en cinq étapes

1. **Reproduisez.** Le problème se produit-il à chaque fois ? Dans quelles conditions exactement ?
2. **Lisez le message.** Ouvrez la console. Notez le fichier, la ligne et le texte de l'erreur.
3. **Isolez.** Commentez ou retirez des morceaux de code jusqu'à trouver la partie responsable.
4. **Formulez une hypothèse**, une seule, et testez-la. Changez une seule chose à la fois.
5. **Comparez** avec la dernière version qui fonctionnait, grâce à `git diff`.

Les problèmes les plus fréquents

SYMPTOME	OU REGARDER EN PREMIER
La page ne change pas après modification	Le fichier est-il enregistré ? Est-ce le bon fichier ouvert dans le navigateur ?
Le CSS ne s'applique pas du tout	Le `href` du `<link>` correspond-il exactement au nom du fichier ?
Une règle CSS ne s'applique pas	Point-virgule manquant ? Faute dans le nom de la classe ? Règle plus précise ailleurs ?
Rien ne se passe au clic	Console : une erreur ? Le script est-il relié ? Le sélecteur trouve-t-il l'élément ?
Cannot read properties of null	`querySelector` n'a rien trouvé : vérifiez le sélecteur, le `#` ou le `.`
is not defined	Faute de frappe dans un nom de variable ou de fonction, majuscule comprise
Le formulaire recharge la page	`event.preventDefault()` est-il bien appelé ?
`git push` est refusé	Faites d'abord `git pull` pour récupérer les changements distants

SYMPTOME	OU REGARDER EN PREMIER
Le site GitHub Pages affiche une erreur 404	Le fichier s'appelle-t-il bien `index.html`, à la racine du dépôt ?

> **TOUJOURS BLOQUE ?**
>
> Après avoir suivi ces étapes, écrivez-moi sur WhatsApp avec le chapitre concerné, ce que vous essayez de faire, ce qui se passe à la place, le message d'erreur exact et ce que vous avez déjà essayé. Vous constaterez que, très souvent, rédiger cette question vous fera trouver la réponse par vous-même.



À PROPOS DE L'AUTEUR

David Mbula Wawina

David Mbula Wawina est développeur Full-Stack, consultant digital et fondateur d'Optimum Entreprise à Kisangani, en République démocratique du Congo. Son parcours s'est construit au contact direct des projets numériques : conception d'applications web, structuration d'architectures techniques, accompagnement de clients et résolution de problèmes concrets de développement.

À travers son expérience, il défend une conviction simple : un développeur ne devient pas professionnel parce qu'il produit beaucoup de code, mais parce qu'il comprend ce qu'il produit, sait identifier une erreur et peut reprendre la main lorsque les outils échouent.

Dans un contexte où l'intelligence artificielle accélère radicalement la production logicielle, il refuse deux extrêmes : rejeter l'IA ou s'y abandonner aveuglément. Son approche consiste à faire de l'IA un accélérateur, tandis que le développeur conserve la compréhension, le jugement et la responsabilité.

Comprendre d'abord. Construire ensuite. Utiliser l'IA pour aller plus vite, jamais pour penser à sa place.

LA SERIE COMPLETE

L'avantage du développeur

Trois livres pour progresser du premier fichier HTML jusqu'au travail professionnel avec l'intelligence artificielle, sans jamais abandonner votre raisonnement, votre créativité ni votre responsabilité.

01 **Penser avant de générer** · vous êtes ici

Apprendre le Web en partant de zéro : HTML, CSS, JavaScript, Git et GitHub. Votre premier portfolio en ligne, construit et compris ligne par ligne.

02 **Construire avec plus de méthode**

Les fondations du développeur professionnel : environnement, back-end, données, lisibilité, tests, sécurité et mise en production.

03 **Rester indispensable à l'ère de l'IA**

Co-piloter l'intelligence artificielle avec méthode : demander, déléguer, auditer, corriger et conserver la responsabilité des décisions.

UNE QUESTION SUR CE LIVRE ?

Un exercice qui bloque, une notion qui reste floue, une erreur que vous ne comprenez pas ? Écrivez-moi en précisant le tome, le chapitre et ce que vous avez déjà essayé. Une question précise reçoit une réponse précise.

Écrire à David sur WhatsApp →

Portfolio : davidwawina.site

TOME 01 · PENSER AVANT DE GÉNÉRER

L'IA peut écrire du code. Vous, vous allez apprendre à le comprendre.

Ce livre s'adresse à celles et ceux qui partent de zéro, même avec un simple smartphone. Chaque notion suit la même logique : une définition simple, une explication, un exemple, puis de la pratique.

- Construire une vraie page avec **HTML**, la rendre belle et responsive avec **CSS**.
- Lui donner vie avec **JavaScript** : clics, formulaires, thème clair et sombre.
- Suivre l'histoire de votre code avec **Git** et le publier sur **GitHub**.
- Un parcours de 30 jours et un **portfolio en ligne** à la fin du livre.

DAVID MBULA WAWINA

L'AVANTAGE DU DÉVELOPPEUR